# Séance 1 — Les bases du langage

**Sommaire**


0. 🛠️ Préparer son poste
   - 0.1 🛠️ Installer Java 25 et Maven avec SDKMAN (macOS, Linux, WSL)
   - 0.2 🛠️ Sous Windows
   - 0.3 🛠️ Installer et configurer IntelliJ IDEA
   - 0.4 🛠️ Le débogueur : regarder son programme s'exécuter
   - 0.5 🛠️ Les raccourcis IntelliJ qui changent la vie
1. Du code source à l'exécution
   - 1.1 🟢 JDK, bytecode et JVM
   - 1.2 🟢 Premier programme : le *compact source file*
   - 1.3 🟢 La forme classique : `public class` et `public static void main`
   - 1.4 🟢 Pourquoi la forme classe dans un projet
   - 1.5 🔵 Un fichier brouillon pour tester une idée
   - 1.6 🛠️ À vous
2. Variables, types et opérateurs
   - 2.1 🟢 Variables et types primitifs
   - 2.2 🟢 Types références, `String` et `null`
   - 2.3 🟢 `var`
   - 2.4 🟢 Opérateurs utiles
   - 2.5 🟢 `==` contre `equals`
   - 2.6 🔒 Débordement d'entier
   - 2.7 🔵 Conversions et constantes
   - 2.8 🛠️ À vous
3. Conditions, boucles, tableaux, méthodes
   - 3.1 🟢 `if` / `else`
   - 3.2 🟢 `switch` en expression
   - 3.3 🟢 Boucles `for`, `while` et for-each
   - 3.4 🟢 Tableaux
   - 3.5 🟢 Méthodes `static`
   - 3.6 🔵 Surcharge
   - 3.7 🔵 Portée des variables
   - 3.8 🟢 Aperçu : lire une `List` et une `Map`
   - 3.9 🛠️ À vous
4. Chaînes de caractères et validation
   - 4.1 🟢 Méthodes de `String`
   - 4.2 🟢 `StringBuilder` : construire un ticket
   - 4.3 🔒 Valider les entrées

- 4.4 🔵 Text blocks
- 4.5 🔵 `import module` (JEP 511)
- 4.6 🟢 Convertir un texte en nombre : `Long.parseLong` et ses exceptions
- 4.7 🔵 Caractères et octets : UTF-8
- 4.8 🛠️ À vous

5. Maven et JUnit
   - 5.1 🟢 Pourquoi Maven ?
   - 5.2 🟢 Structure d'un projet
   - 5.3 🟢 Le `pom.xml` minimal (Java 25)
   - 5.4 🟢 Les commandes Maven
   - 5.5 🟢 Premier test JUnit
   - 5.6 🟢 Écrire de bons tests
   - 5.7 🔵 Tests paramétrés et tests d'exception
   - 5.8 🛠️ À vous
6. 🟣 Défis pour ceux qui connaissent déjà
   - 6.1 🟣 Défi 1 — Algorithme de Luhn (numéro de carte)
   - 6.2 🟣 Défi 2 — Robustesse d'un mot de passe
   - 6.3 🟣 Défi 3 — Chiffre de César
   - 6.4 🟣 Défi 4 — FizzBuzz piloté par les tests (TDD)
   - 6.5 🟣 Défi 5 — `switch` à motifs sur un `record`
   - 6.6 🟣 Défi 6 — Générer un code promo imprévisible
   - 6.7 🔵 Java moderne : si vous avez connu Java 8 ou 11
   - 6.8 🟣 Sous le capot : quatre manipulations
   - 6.9 🟣 Défi 7 — Parler le protocole Redis à la main
7. Ce qu'il faut retenir
8. Pour aller plus loin

Annexe A — Solutions des défis

Annexe B — 🛠️ Lire les erreurs : les plus fréquentes

Annexe C — 🛠️ Pense-bête : vous venez de Python ou de C

> **Objectifs** : AA1 — Écrire, compiler et exécuter un programme Java (types, conditions, boucles, méthodes), puis le construire et le tester avec Maven et JUnit. **Prérequis** : aucun en Java. Avoir déjà écrit un algorithme simple (une boucle, une condition) dans n'importe quel langage aide. Installer le **JDK 25** et **Maven 3.9** (ou plus récent) avant la séance. **Comment lire ce support** : 🟢 indispensable · 🔵 pour approfondir · 🟣 bonus · 🔒 sécurité · 🛠️ atelier, à faire sur votre machine

Ce support accompagne les slides de la séance 1. Il contient tout ce qui a été dit en cours, avec plus de détails, des exemples complets que vous pouvez copier et exécuter, et des défis 🟣 pour celles et ceux qui connaissent déjà Java.

**Deux parcours.** Vous découvrez Java ? Installez votre poste (partie 0), puis faites l'atelier 🛠️ **À vous** qui termine chaque partie : c'est en tapant et en exécutant le code qu'on apprend. Gardez l'annexe B (les erreurs fréquentes) et l'annexe C (pense-bête Python et C) sous la main. Vous connaissez déjà Java ? Survolez les parties 1 à 5, faites l'atelier du §5.8, puis passez à la partie 6 : ce qui a changé depuis Java 8 (§6.7), ce qui se passe sous le capot (§6.8) et des défis.

**Une convention pour toute la séance.** Nous ne manipulons pas encore d'objets. Pour représenter un prix, nous utilisons donc un **entier en centimes** : `1250` signifie 12,50 €. C'est un choix volontaire et temporaire. Les nombres à virgule (`double`) sont approximatifs et ne conviennent pas à l'argent. En séance 2, nous passerons à `BigDecimal`, le type adapté aux montants.

## 0. 🛠️ Préparer son poste

Cette partie se fait **sur votre machine**, avant la séance ou pendant les premières minutes. À la fin, vous avez trois choses : Java 25 et Maven installés, IntelliJ IDEA configuré, et un premier programme qui tourne, que vous savez suivre pas à pas avec le débogueur.

Pourquoi tant d'insistance ? En programmation, on apprend **en tapant du code et en le faisant tourner**, pas en lisant. Plus vite votre poste est prêt, plus vite vous pouvez essayer chaque exemple de ce support.

## 0.1 🛠️ Installer Java 25 et Maven avec SDKMAN (macOS, Linux, WSL)

**Pourquoi SDKMAN ?** Vous aurez besoin de plusieurs versions de Java au cours de votre carrière : Java 25 pour ce cours, Java 17 ou 21 pour un projet d'entreprise… SDKMAN installe ces versions dans votre dossier personnel, **sans droits administrateur**, et vous laisse passer de l'une à l'autre en une commande. Il installe aussi Maven.

**Étape 1 — Installer SDKMAN.** Dans un terminal :

```
$ curl -s "https://get.sdkman.io" | bash
$ source "$HOME/.sdkman/bin/sdkman-init.sh"
$ sdk version
SDKMAN!
script: 5.x.x
```

> 🔒 **Sécurité** — `curl … | bash` exécute un script téléchargé sans que vous l'ayez lu. C'est pratique, mais cela revient à faire confiance au site. Ici, la source est le site officiel de SDKMAN, en HTTPS. Sur une machine sensible, téléchargez d'abord le script ( `curl -s https://get.sdkman.io -o install.sh` ), lisez-le, puis lancez-le.

**Étape 2 — Choisir une version de Java.** Listez les distributions disponibles :

```
$ sdk list java
 Vendor        | Use | Version      | Dist | Status | Identifier
 Temurin       |     | 25.0.5.0+8   | tem  |        | 25.0.5.0+8-tem
 ...
```

Cherchez une ligne **Temurin** (la distribution libre de la fondation Eclipse) en version **25**, et copiez la valeur de la colonne **Identifier**. Le numéro exact change à chaque mise à jour de sécurité : prenez le plus récent en 25.

**Étape 3 — Installer Java et Maven.**

```
$ sdk install java 25.0.5.0+8-tem   ← remplacez par l'identifiant copié
Do you want java 25.0.5.0+8-tem to be set as default? (Y/n): Y
$ sdk install maven
```

**Étape 4 — Vérifier.** Ouvrez un **nouveau** terminal, puis :

```
$ java -version
openjdk version "25.0.5" … LTS
$ javac -version
javac 25.0.5
$ mvn -v
Apache Maven 3.9.x
Java version: 25.0.5
```

Les trois lignes importantes : `java` en 25, `javac` en 25, et Maven qui utilise **Java 25** (ligne `Java version`).

Quelques commandes SDKMAN utiles :

| Commande | Effet |
|---|---|
| `sdk current java` | affiche la version active |
| `sdk use java <identifiant 21>` | change de version **dans ce terminal seulement** |
| `sdk default java <identifiant 25>` | change la version par défaut |
| `sdk upgrade` | liste les mises à jour disponibles |

**Problèmes fréquents.**

| Symptôme | Cause | Solution |
|---|---|---|
| `sdk: command not found` | le terminal a été ouvert avant l'installation | ouvrir un nouveau terminal, ou `source "$HOME/.sdkman/bin/sdkman-init.sh"` |
| `java -version` affiche 17 ou 21 | une autre version est la version par défaut | `sdk default java <identifiant 25>` |
| `mvn -v` affiche une autre version de Java | la variable `JAVA_HOME` pointe ailleurs | retirer l'ancien `export JAVA_HOME=…` de `~/.zshrc` ou `~/.bashrc` : SDKMAN la gère |
| `javac: command not found` alors que `java` marche | un JRE ou un ancien Java système passe avant | vérifier avec `which javac` ; il doit être dans `~/.sdkman/candidates/java/current/bin` |

## 0.2 🛠️ Sous Windows

SDKMAN ne fonctionne pas directement sous Windows. Deux options :

1. **Recommandé : WSL** (*Windows Subsystem for Linux*). Dans un PowerShell administrateur : `wsl --install`, puis redémarrez. Vous obtenez un vrai terminal Linux (Ubuntu), dans lequel vous suivez le §0.1 à l'identique. C'est aussi l'environnement le plus proche des serveurs sur lesquels votre code tournera.
2. **Sans terminal Linux : tout par IntelliJ.** IntelliJ sait télécharger un JDK (§0.3, étape 3) et embarque son propre Maven. Pour la ligne de commande Windows, installez Java avec `winget install EclipseAdoptium.Temurin.25.JDK` dans un PowerShell, puis ouvrez un nouveau terminal et vérifiez `java -version`.

Avec l'option 2, les commandes `mvn` de ce support se lancent depuis IntelliJ (fenêtre **Maven**, à droite) plutôt que dans un terminal.

## 0.3 🛠️ Installer et configurer IntelliJ IDEA

**Pourquoi un IDE ?** Un éditeur de texte suffit pour écrire du Java, mais un IDE (*Integrated Development Environment*) vous signale les erreurs **pendant que vous tapez**, complète le code, lance les programmes et les tests en un clic, et surtout fournit un **débogueur**.

**Étape 1 — Installer.** Téléchargez **IntelliJ IDEA** sur jetbrains.com/idea. Les fonctions utilisées dans ce cours sont gratuites. En tant qu'étudiant, vous pouvez aussi demander une licence éducation gratuite qui débloque tout (jetbrains.com/student).

**Étape 2 — Créer un projet.** *New Project* :

| Champ | Valeur |
|---|---|
| Name | `seance1` |
| Language | Java |
| Build system | **Maven** |
| JDK | **25** (voir étape 3 s'il n'apparaît pas) |
| Add sample code | coché |
| Advanced Settings → GroupId | `fr.foodfast` |

IntelliJ crée la structure Maven (`pom.xml`, `src/main/java`, `src/test/java`) et une classe `Main`.

**Étape 3 — Si le JDK 25 n'apparaît pas.** Dans la liste *JDK* :

- avec SDKMAN : *Add JDK from disk…* et choisissez `~/.sdkman/candidates/java/current` (sous WSL : *Add JDK from disk* puis le chemin `\\wsl$\Ubuntu\home\<vous>\.sdkman\candidates\java\current`) ;
- sans SDKMAN : *Download JDK…*, version **25**, vendeur **Eclipse Temurin**.

**Étape 4 — Lancer.** Ouvrez `Main.java` : une flèche verte ▶ apparaît dans la marge, à gauche de `main`. Cliquez dessus, puis *Run*. La sortie s'affiche dans la fenêtre **Run**, en bas.

**Deux raccourcis de saisie à connaître dès maintenant** : tapez `main` puis Tab pour générer une méthode `main`, et `sout` puis Tab pour générer `System.out.println();`.

**Piège.**

❌ Créer le fichier dans le mauvais dossier. Une classe placée à la racine du projet ou dans `src/test/java` ne sera pas lancée comme vous le pensez.

✅ Le code de l'application va dans **`src/main/java`**, les tests dans **`src/test/java`** (§5.2). Clic droit sur `src/main/java` → *New* → *Java Class*.

## 0.4 🛠️ Le débogueur : regarder son programme s'exécuter

**Pourquoi ?** Quand un programme donne un mauvais résultat, la tentation est d'ajouter des `println` partout. Le débogueur fait mieux : il **arrête** le programme sur une ligne choisie, vous montre la valeur de **toutes** les variables, puis avance ligne par ligne. C'est l'outil qui vous fera gagner le plus de temps cette année.

**Exemple : un total faux.** Ce programme compile, s'exécute… et donne un mauvais total. Copiez-le dans IntelliJ :

```java
// Fichier : BuggyTotal.java
public class BuggyTotal {

    static int totalCents(int[] pricesCents) {
        int total = 0;
        for (int i = 1; i < pricesCents.length; i++) {
            total = total + pricesCents[i];
        }
        return total;
    }

    public static void main(String[] args) {
        int[] basket = {1250, 450, 800};   // 12,50 € + 4,50 € + 8,00 €
        int total = totalCents(basket);
        System.out.println("Total : " + total + " centimes");  // attendu :
2500
    }
}
```

```
Total : 1250 centimes
```

**Trouver le bug avec le débogueur, en cinq gestes :**

1. **Poser un point d'arrêt** : cliquez dans la marge, sur le numéro de la ligne `int total = 0;`. Un rond rouge apparaît.
2. **Lancer en mode debug** : flèche verte ▶ à côté de `main` → *Debug* (et non *Run*). Le programme s'arrête sur la ligne marquée, surlignée en bleu.
3. **Observer** : la fenêtre **Debug**, en bas, affiche les variables (`pricesCents`, puis `total`, `i`…). Dépliez `pricesCents` pour voir ses trois cases.
4. **Avancer d'une ligne** : *Step Over* (F8). Avancez jusqu'à entrer dans la boucle et regardez la valeur de `i` au premier tour. Elle vaut `1` : la case `0` (le 12,50 €) n'est jamais additionnée.
5. **Reprendre** : *Resume* (F9 sous Windows et Linux, ⌥⌘R sous macOS) laisse le programme finir.

Corrigez (`int i = 0`), relancez : `Total : 2500 centimes`.

| Action | Windows / Linux | macOS |
|---|---|---|
| Poser ou retirer un point d'arrêt | Ctrl+F8 | ⌘F8 |
| Avancer d'une ligne (*Step Over*) | F8 | F8 |

| Action | Windows / Linux | macOS |
|---|---|---|
| Entrer dans la méthode appelée (*Step Into*) | F7 | F7 |
| Reprendre jusqu'au prochain point d'arrêt | F9 | ⌥⌘R |
| Évaluer une expression (*Evaluate*) | Alt+F8 | ⌥F8 |

Sur un Mac, les touches F7 et F8 demandent souvent la touche `fn`.

> 🔵 **Évaluer une expression.** Pendant que le programme est arrêté, *Evaluate Expression* (Alt+F8) vous laisse taper n'importe quelle expression Java, par exemple `pricesCents[0] + pricesCents[1]`, et affiche le résultat. Pratique pour vérifier une intuition sans relancer le programme.

## 0.5 🛠️ Les raccourcis IntelliJ qui changent la vie

| Action | Windows / Linux | macOS |
|---|---|---|
| Complétion | Ctrl+Espace | ^Espace |
| **Corriger l'erreur sous le curseur** (importer, créer la méthode…) | Alt+Entrée | ⌥↩ |
| Reformater le fichier | Ctrl+Alt+L | ⌥⌘L |
| Renommer partout | Maj+F6 | ⇧F6 |
| Chercher n'importe quoi (fichier, classe, action) | Maj Maj | ⇧⇧ |
| Aller à la définition | Ctrl+clic | ⌘clic |

Le plus utile : **Alt+Entrée** sur un mot souligné en rouge. IntelliJ propose presque toujours la bonne correction… mais lisez-la avant de l'accepter.

---

# 1. Du code source à l'exécution

## 1.1 🟢 JDK, bytecode et JVM

**Pourquoi ?** Un programme écrit en C est compilé en code machine pour **un** processeur et **un** système. Pour le faire tourner ailleurs, il faut le recompiler. Java a été conçu pour éviter cela : « *write once, run anywhere* ».

**Comment ?** Le passage du code source à l'exécution se fait en deux temps :

```
Hello.java  ──javac──→  Hello.class  ──java──→  JVM
(code source,           (bytecode,              (exécute le bytecode
 lisible par vous)       portable)               sur votre machine)
```

1. **Compilation.** Le compilateur `javac` lit votre fichier `.java`, vérifie qu'il est correct (syntaxe, types) et produit un fichier `.class` qui contient du **bytecode**. Le bytecode est un langage intermédiaire : ce n'est pas du code machine, et il est identique sur Windows, Linux et macOS.
2. **Exécution.** La commande `java` démarre une **JVM** (*Java Virtual Machine*, machine virtuelle Java). La JVM charge le bytecode, le **vérifie**, puis l'exécute. Elle commence par l'interpréter, puis compile en code machine les parties exécutées souvent : c'est la compilation **JIT** (*Just-In-Time*, « juste à temps »). Java n'est donc ni purement compilé, comme C, ni purement interprété, comme Python.

La JVM gère aussi la mémoire. Les objets que vous créez sont placés dans une zone appelée le **tas** (*heap*). Quand un objet n'est plus utilisé par personne, le **ramasse-miettes** (*garbage collector*, GC) le supprime. Vous n'avez jamais à libérer la mémoire vous-même, contrairement au C.

**Ce que vous installez : le JDK.** Le **JDK** (*Java Development Kit*) contient la JVM et les outils du développeur :

| Outil | Rôle |
|---|---|
| `javac` | compile les `.java` en `.class` |
| `java` | lance la JVM ; sait aussi exécuter directement un fichier `.java` |
| `jar` | regroupe des `.class` dans une archive `.jar` |
| `javadoc` | génère la documentation HTML à partir des commentaires |

| Outil | Rôle |
|---|---|
| `javap` | affiche le bytecode d'un `.class` (`javap -c Hello`) |

Vous rencontrerez peut-être le terme **JRE** (*Java Runtime Environment*) : c'était autrefois une version allégée, sans les outils de développement, installée chez les utilisateurs. Depuis Java 11, il n'existe plus de JRE séparé chez Oracle et OpenJDK. Aujourd'hui, on livre une application avec un environnement d'exécution taillé sur mesure (`jlink`), un installeur (`jpackage`) ou une image Docker.

**Quelle version ?** Ce cours utilise **Java 25**, une version **LTS** (*Long-Term Support*, support à long terme) sortie en septembre 2025. Les entreprises restent en général sur une LTS (17, 21, 25) pendant plusieurs années. Vérifiez votre installation :

```
$ java -version
openjdk version "25.0.3" 2026-04-21 LTS
$ javac -version
javac 25.0.3
```

**Piège : donner un nom de fichier à `java`.**

❌ `java Hello.class` : la commande `java` attend un **nom de classe**, pas un fichier compilé.

```
$ java Hello.class
Erreur : impossible de trouver ou de charger la classe principale Hello.class
```

✅ `java Hello` (nom de la classe, sans extension) après `javac Hello.java`, ou `java Hello.java` (fichier **source**) pour tout faire en une commande.

> 🔒 **Sécurité** — Java élimine par construction une famille entière de failles du C : pas d'arithmétique de pointeurs, contrôle systématique des indices de tableau, mémoire libérée automatiquement, bytecode vérifié au chargement. Un débordement de tampon (*buffer overflow*) classique n'est pas possible en Java pur. Cela ne rend pas un programme Java sûr pour autant : les failles logiques (débordement d'entier §2.6, entrées non validées §4.3, injection SQL en séance 4) restent possibles. Pensez aussi à mettre à jour votre JDK : Oracle et OpenJDK publient des correctifs de sécurité quatre fois par an.

## 1.2 🟢 Premier programme : le *compact source file*

**Pourquoi ?** Pendant vingt-cinq ans, le premier programme Java ressemblait à ceci :

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Bonjour");
    }
}
```

Cinq notions (`public`, `class`, `static`, `void`, `String[]`) avant d'afficher une ligne. Depuis **Java 25** (JEP 512, fonctionnalité finale), on peut écrire beaucoup plus simple.

**Comment ?** Un *compact source file* (fichier source compact) est un fichier `.java` qui contient directement des méthodes, sans déclarer de classe. Le point d'entrée est une méthode `main` sans paramètre :

```
// Fichier : Hello.java
void main() {
    IO.println("Bienvenue chez FoodFast !");
    IO.println("Votre commande est en préparation.");
}
```

On le compile, puis on l'exécute :

```
$ javac Hello.java        → crée Hello.class
$ java Hello              → exécute le bytecode
Bienvenue chez FoodFast !
Votre commande est en préparation.
```

Ou, en une seule commande, pour un programme d'un seul fichier :

```
$ java Hello.java         → compile en mémoire, puis exécute
```

Quelques points à retenir :

- **`void main()`** : `main` est le nom imposé du point d'entrée. `void` signifie que la méthode ne renvoie aucune valeur. Les accolades `{ … }` délimitent son **corps**.
- **`IO.println(...)`** affiche une valeur puis passe à la ligne. `IO.print(...)` affiche sans passer à la ligne. `IO` est une classe du paquetage `java.lang`, toujours disponible sans `import`. Elle est apparue en Java 25.

- Chaque **instruction** se termine par un point-virgule `;`.
- Un *compact source file* importe automatiquement toutes les classes publiques du module `java.base` : `List`, `Map`, `Random`, `LocalDate`… sont utilisables sans `import`.
- Vous rencontrerez partout `System.out.println(...)` : c'est l'écriture historique, qui fait la même chose que `IO.println(...)`.

**Lire une erreur de compilation.** Oubliez le `;` de la première ligne et compilez :

```
$ javac Hello.java
Hello.java:2: error: ';' expected
    IO.println("Bienvenue chez FoodFast !")
                                           ^
1 error
```

Le compilateur donne le **fichier**, la **ligne** (`:2`) et la position (`^`). Prenez l'habitude de lire la **première** erreur : les suivantes en sont souvent la conséquence.

**Piège.**

❌ Un appel de méthode en dehors de toute méthode :

```
IO.println("Bonjour");    // ne compile pas : une instruction doit être dans une méthode
void main() { }
```

✅ Les instructions vont **dans** le corps de `main` (ou d'une autre méthode), comme dans `Hello.java` ci-dessus.

## 1.3 🟢 La forme classique : `public class` et `public static void main`

**Pourquoi ?** C'est la forme que vous trouverez dans la quasi-totalité du code existant, des tutoriels, des frameworks et des projets d'entreprise. Il faut savoir la lire et l'écrire.

**Comment ?**

```java
// Fichier : FoodFastApp.java
public class FoodFastApp {
    public static void main(String[] args) {
        System.out.println("Bienvenue chez FoodFast !");
        IO.println("Nombre d'arguments reçus : " + args.length);
    }
}
```

Décomposons chaque mot :

| Élément | Signification |
|---|---|
| `public class FoodFastApp` | déclare une **classe** nommée `FoodFastApp`. Une classe publique doit se trouver dans un fichier du **même nom** : `FoodFastApp.java`. |
| `public` | la méthode est accessible depuis n'importe quelle autre classe (la JVM doit pouvoir l'appeler). |
| `static` | la méthode appartient à la **classe** : on l'appelle sans créer d'objet. |
| `void` | la méthode ne renvoie rien. |
| `main` | nom imposé du point d'entrée. |
| `String[] args` | les **arguments** passés en ligne de commande, sous forme de tableau de chaînes (§3.4). |

```
$ java FoodFastApp.java pizza 2
Bienvenue chez FoodFast !
Nombre d'arguments reçus : 2
```

Pour l'instant, retenez qu'une **classe** est un conteneur nommé qui regroupe des méthodes. Nous verrons en séance 2 qu'elle sert surtout à décrire des **objets** (une commande, un plat…).

**Conventions de nommage**, à respecter dès maintenant :

| Élément | Convention | Exemple |
|---|---|---|
| Classe | `UpperCamelCase` | `OrderCalculator` |
| Méthode, variable | `lowerCamelCase` | `totalCents`, `isValidQuantity` |

| Élément | Convention | Exemple |
|---|---|---|
| Constante (`static final`) | `SCREAMING_SNAKE_CASE` | `MAX_QUANTITY` |
| Package | minuscules, nom de domaine inversé | `fr.foodfast` |

**Piège.**

❌ Le nom du fichier ne correspond pas à la classe publique :

```
// Fichier Main.java
public class FoodFastApp { … }   // erreur : class FoodFastApp is public,
                                 // should be declared in a file named
FoodFastApp.java
```

✅ Renommez le fichier `FoodFastApp.java`, ou la classe `Main`.

## 1.4 🟢 Pourquoi la forme classe dans un projet

Un *compact source file* déclare en réalité une classe **implicite** : le compilateur crée une classe pour vous, mais **votre code ne peut pas la désigner par son nom**. Conséquence directe : aucune autre classe ne peut appeler ses méthodes. En particulier, une classe de test JUnit (§5.5) ne peut pas les tester.

| | *Compact source file* | Classe nommée |
|---|---|---|
| Usage | apprendre, petit script, prototype | projet, bibliothèque, application |
| Appelable depuis une autre classe | non | oui : `OrderCalculator.totalCents(...)` |
| Testable avec JUnit | non | oui |
| Rangée dans un package Maven | non | oui |

Dans ce support, les petits exemples sont des *compact source files*, plus lisibles. Dès qu'une méthode mérite un test (§5), elle va dans une **classe nommée**.

> 🔵 **Pour être précis.** Java 25 autorise aussi un `void main()` sans `static` ni `String[] args` dans une classe nommée. La JVM crée alors un objet de la classe et appelle sa méthode `main`. Ce qui compte pour les tests, c'est donc la **classe nommée**, pas la forme de `main`. Nous gardons `public static void main(String[] args)` dans les classes nommées parce que c'est la forme que vous lirez partout.

**Une règle pour tout ce support.** Nos méthodes utilitaires sont toujours déclarées `static`, y compris dans les *compact source files* (où ce n'est pas obligatoire). Ainsi, vous pouvez les copier telles quelles dans une classe nommée, et les appeler depuis un `public static void main` ou depuis un test.

## 1.5 🔵 Un fichier brouillon pour tester une idée

Pour vérifier une intuition (« que vaut `7 / 2` ? »), gardez un fichier brouillon : un *compact source file* que vous modifiez et relancez à volonté, avec ▶ dans IntelliJ ou `java Brouillon.java` dans un terminal (sans `javac`).

```
// Fichier : Brouillon.java
void main() {
    IO.println(7 / 2);                      // 3
    IO.println("promo10".toUpperCase());    // PROMO10
    IO.println(Integer.MAX_VALUE + 1);      // -2147483648
}
```

Une ligne qui lève une exception arrête le programme : placez-la en dernier, ou commentez-la après l'avoir vue.

## 1.6 🛠️ À vous

> ⏱️ 15 à 20 min, sur votre machine. Les solutions sont repliées : essayez d'abord, ouvrez ensuite.

**Exercice 1 — Compiler à la main et lire trois erreurs** (⏱️ 8 min)

1. Dans un dossier vide, créez `Hello.java` avec le programme du §1.2.
2. Dans un terminal, placé dans ce dossier : `javac Hello.java`, puis `ls` (ou `dir` sous Windows). Quel fichier est apparu ?
3. Lancez `java Hello`, puis `java Hello.java`. Quelle différence ?

4. Introduisez **une** erreur à la fois, compilez, lisez le message, puis réparez :
    - supprimez le `;` à la fin d'une ligne ;
    - écrivez `IO.printn` au lieu de `IO.println` ;
    - supprimez le `"` fermant d'un texte.

Pour chaque erreur, notez la **ligne** indiquée et le **message**.

▶ Solution de l'exercice 1

**Exercice 2 — La même chose dans IntelliJ** (⏱ 8 min)

Dans le projet créé au §0.3, créez une classe `FoodFastApp` : clic droit sur `src/main/java` → *New* → *Java Class*. Recopiez le programme du §1.3.

1. Lancez-la avec le ▶ vert dans la marge, à côté de `main`. La sortie s'affiche en bas, dans la fenêtre *Run*.
2. Changez le message, relancez.
3. Supprimez un `;`. IntelliJ souligne l'erreur en rouge **avant même de compiler**. Survolez-la : c'est le même message que `javac`.

**Exercice 3** 🔵 **— Regarder le bytecode** (⏱ 5 min)

Après `javac Hello.java`, lancez `javap -c Hello`. Retrouvez dans la sortie la chaîne `"Bienvenue chez FoodFast !"` et l'appel à `println`. Vous lisez ce que la JVM exécute réellement.

**Exercice 4** 🔵 **— Accueillir le restaurant passé en argument** (⏱ 10 min)

Le `String[] args` de `main` contient les mots tapés après le nom du programme : `java FoodFastApp "Chez Mario"` donne `args[0]` égal à `"Chez Mario"`. Cet exercice utilise un tableau et un `if`, vus en partie 3 : à garder pour plus tard si c'est nouveau pour vous.

Modifiez `FoodFastApp` : elle affiche `Bienvenue chez <restaurant> !`, où le restaurant est le premier argument. Sans argument, elle affiche `Bienvenue chez FoodFast !`.

1. Lancez-la avec ▶ : vous devez voir `Bienvenue chez FoodFast !`.
2. Passez un argument : menu *Run* → *Edit Configurations…* → champ *Program arguments* : `"Chez Mario"` (avec les guillemets, sinon ce sont deux arguments). Relancez.
3. Que se passe-t-il si vous lisez `args[0]` sans vérifier qu'il existe, et sans argument ?

**Indice.** `args.length` donne le nombre d'arguments.

▶ Solution de l'exercice 4

## 2. Variables, types et opérateurs

### 2.1 🟢 Variables et types primitifs

**Pourquoi ?** Un programme manipule des données : une quantité, un prix, un nom de plat. Pour les conserver, on utilise des **variables**. Java est un langage à **typage statique** : chaque variable a un **type** fixé à la compilation, et le compilateur refuse de mélanger les types. Beaucoup d'erreurs sont ainsi détectées avant même d'exécuter le programme.

**Comment ?** On **déclare** une variable avec son type et son nom, et on lui **affecte** une valeur avec `=` :

```
int quantity;          // déclaration
quantity = 3;          // affectation
int fee = 199;         // déclaration + initialisation
quantity = quantity + 1;  // nouvelle valeur : 4
```

Java possède **huit types primitifs**. Cinq suffisent pour ce cours :

| Type | Contient | Bornes | Exemple |
|---|---|---|---|
| `int` | entier sur 32 bits | –2 147 483 648 à 2 147 483 647 | `int quantity = 3;` |
| `long` | entier sur 64 bits | environ ±9,2 × $10^{18}$ | `long orderCount = 12_000_000_000L;` |
| `double` | nombre à virgule flottante, 64 bits | très grand, mais **approché** | `double rating = 4.5;` |
| `boolean` | vrai ou faux | `true`, `false` | `boolean delivered = false;` |
| `char` | un caractère Unicode (16 bits) | `'\u0000'` à `'￿'` | `char size = 'M';` |

Les trois autres, `byte` (8 bits), `short` (16 bits) et `float` (32 bits), servent surtout pour des données binaires ou des calculs très spécialisés.

Détails utiles :

- Un nombre écrit dans le code (un **littéral**) est un `int` par défaut. Ajoutez `L` pour un `long` (`12_000_000_000L`).
- Le tiret bas `_` améliore la lisibilité des grands nombres : `1_000_000`.
- Un `char` s'écrit entre apostrophes (`'M'`), une chaîne entre guillemets (`"M"`). Ce ne sont pas les mêmes types.

```
// Fichier : Types.java
void main() {
    int quantity = 3;
    int unitPriceCents = 1250;               // 12,50 € en centimes
    long orderCount = 12_000_000_000L;        // trop grand pour un int
    double rating = 4.5;
    boolean delivered = false;
    char size = 'M';

    int totalCents = quantity * unitPriceCents;
    IO.println("Total : " + totalCents + " centimes");   // Total : 3750
centimes
    IO.println("Commandes : " + orderCount);
    IO.println("Note : " + rating + ", livrée : " + delivered + ", taille : " +
size);
    IO.println("Plus grand int : " + Integer.MAX_VALUE);  // 2147483647
}
```

**Piège : `double` pour l'argent.**

❌ Les `double` sont stockés en binaire. Beaucoup de nombres décimaux, comme 0,1, n'ont pas de représentation exacte :

```
// Fichier : DoubleMoney.java
void main() {
    double price = 0.1 + 0.2;
    IO.println(price);                  // 0.30000000000000004
    IO.println(price == 0.3);           // false
}
```

✅ Pour cette séance : des **centimes dans un `int`**. Les calculs sur les entiers sont exacts.

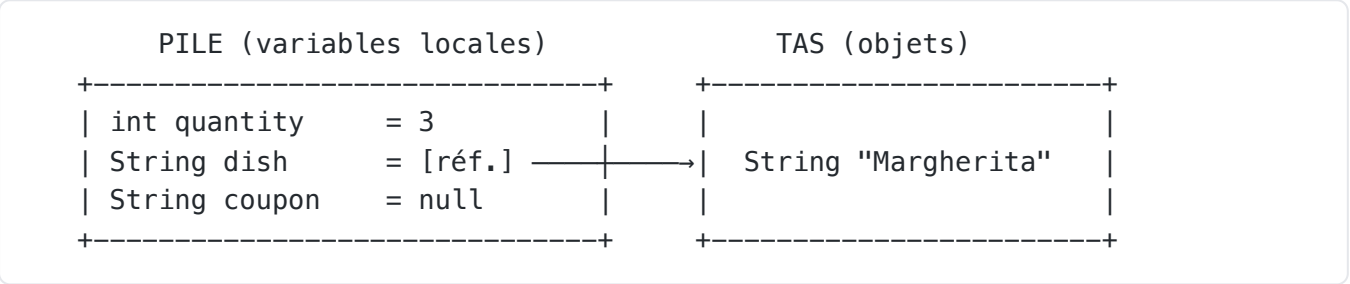
```
// Fichier : CentsMoney.java
void main() {
    int priceCents = 10 + 20;           // 0,10 € + 0,20 €
    IO.println(priceCents);             // 30
    IO.println(priceCents == 30);       // true
}
```

En séance 2, nous utiliserons `BigDecimal`, qui gère aussi les arrondis et les devises.

## 2.2 🟢 Types références, `String` et `null`

**Pourquoi ?** Les primitifs ne contiennent qu'une valeur simple. Pour du texte, une date ou une commande, il faut des **objets**. Une variable de type objet ne contient pas l'objet lui-même : elle contient une **référence**, c'est-à-dire l'adresse de l'objet en mémoire.

**Comment ?** Le type `String` (chaîne de caractères) est le premier type référence que vous utilisez :




- La **pile** (*stack*) contient les variables locales de chaque méthode. Une variable primitive y contient directement sa valeur.
- Le **tas** (*heap*) contient les objets. Une variable référence pointe vers un objet du tas.
- `null` est une valeur spéciale : la variable ne désigne **aucun** objet.

| | **Primitif** | **Référence** |
|---|---|---|
| Nom du type | minuscule : `int`, `double` | majuscule : `String`, `Integer` |
| Contient | la valeur | l'adresse d'un objet |
| Méthodes | aucune | oui : `dish.length()` |
| Peut valoir `null` | non | oui |

Chaque primitif a une classe « enveloppe » (*wrapper*) correspondante : `Integer` pour `int`, `Double` pour `double`, `Boolean`, `Character`, `Long`… Vous les utiliserez en séance 2 avec les collections. Elles fournissent aussi des méthodes utiles comme `Integer.parseInt("42")`.

```java
// Fichier : References.java
void main() {
    String dish = "Margherita";
    IO.println(dish.length());          // 10
    IO.println(dish.toUpperCase());     // MARGHERITA
    String coupon = null;
    IO.println(coupon == null);         // true : on peut tester null avec ==
}
```

**Piège : appeler une méthode sur `null`.**

❌ La variable ne désigne aucun objet : il n'y a donc personne pour exécuter la méthode. Le programme s'arrête sur une `NullPointerException`, l'erreur la plus fréquente en Java.

```java
// Fichier : NullCrash.java (exception attendue)
void main() {
    String coupon = null;
    IO.println(coupon.length());        // NullPointerException
}
```

✅ Tester `null` **avant** d'appeler une méthode :

```java
// Fichier : NullCheck.java
void main() {
    String coupon = null;
    if (coupon != null && !coupon.isBlank()) {
        IO.println("Code promo : " + coupon);
    } else {
        IO.println("Pas de code promo");
    }
}
```

## 2.3 🟢 `var`

**Pourquoi ?** Écrire le type à gauche alors qu'il est évident à droite est répétitif.

**Comment ?** Pour une **variable locale** initialisée dans la même instruction, `var` demande au compilateur de **déduire** le type :

```java
// Fichier : VarDemo.java
void main() {
    var quantity = 3;               // int
    var dish = "Margherita";        // String
    var builder = new StringBuilder();  // StringBuilder
    quantity = quantity + 2;        // toujours un int
    builder.append(dish).append(" x").append(quantity);
    IO.println(builder);            // Margherita x5
}
```

`var` **n'est pas** un type dynamique comme en Python ou JavaScript. Le type est fixé une fois pour toutes à la compilation.

**Piège.**

❌ Ces lignes ne compilent pas :

```java
var quantity = 3;
quantity = "trois";      // int ≠ String : le type de quantity est fixé
var coupon;              // pas de valeur : impossible de déduire le type
var nothing = null;      // null n'a pas de type exploitable
```

✅ Utilisez `var` quand le type est **évident à la lecture** (`var dish = "Margherita";`). Écrivez le type explicitement quand il apporte une information (`int totalCents = computeTotal();`). `var` est interdit pour les paramètres de méthode et les attributs de classe.

## 2.4 🟢 Opérateurs utiles

| Catégorie | Opérateurs | Exemple FoodFast |
|---|---|---|
| Arithmétique | `+` `-` `*` `/` `%` | `int total = price * quantity;` |
| Affectation composée | `+=` `-=` `*=` | `total += fee;` (équivaut à `total = total + fee;`) |
| Incrément | `++` `--` | `count++;` (ajoute 1) |
| Comparaison | `==` `!=` `<` `<=` `>` `>=` | `quantity <= 20` |
| Logique | `&&` (et) `\|\|` (ou) `!` (non) | `quantity >= 1 && quantity <= 20` |

| Catégorie | Opérateurs | Exemple FoodFast |
|---|---|---|
| Conversion (*cast*) | `(type)` | `(double) cents / 100` |

Trois règles suffisent :

1. **Priorité** : `*`, `/`, `%` passent avant `+` et `-`. Dans le doute, mettez des parenthèses ; elles ne coûtent rien et aident le lecteur.
2. **Division entière** : entre deux entiers, `/` donne un entier en **tronquant** vers zéro (`7 / 2` vaut `3`, `-7 / 2` vaut `-3`). Le **modulo** `%` donne le reste (`7 % 2` vaut `1`). Pour obtenir `3.5`, convertissez un des deux opérandes en `double` **avant** la division.
3. **Court-circuit** : `&&` n'évalue pas la partie droite si la gauche est fausse ; `||` ne l'évalue pas si la gauche est vraie. C'est ce qui rend `s != null && s.isBlank()` sûr.

```
// Fichier : Operators.java
void main() {
    IO.println(7 / 2);                 // 3   (division entière)
    IO.println(7 % 2);                 // 1   (reste : 7 est impair)
    IO.println((double) 7 / 2);        // 3.5
    IO.println(-7 / 2);                // -3  (troncature vers zéro)

    int totalCents = 2850;
    IO.println(totalCents / 100 + " € et " + totalCents % 100 + " centimes");
// 28 € et 50 centimes

    int quantity = 5;
    boolean valid = quantity >= 1 && quantity <= 20;
    IO.println("Quantité valide : " + valid);   // true

    String coupon = null;
    boolean hasCoupon = coupon != null && !coupon.isBlank();  // pas
d'exception
    IO.println("Code promo : " + hasCoupon);    // false
}
```

**Piège : `+` entre une chaîne et des nombres.**

❌ L'expression est évaluée de gauche à droite. Dès qu'un opérande est une `String`, `+` devient une **concaténation** :

```
// Fichier : ConcatTrap.java
void main() {
    IO.println("Total : " + 1 + 2);     // Total : 12   ("Total : 1" puis +
"2")
    IO.println(1 + 2 + " €");           // 3 €          (1 + 2 = 3, puis + "
€")
}
```

✅ Mettez le calcul entre parenthèses : `"Total : " + (1 + 2)` affiche `Total : 3`.

## 2.5 🟢 `==` contre `equals`

**Pourquoi ?** Un client tape un code promo. Vous le comparez au code attendu. Avec `==`, la comparaison peut échouer alors que les deux textes sont identiques.

**Comment ?**

- `==` compare le contenu de deux variables. Pour un primitif, c'est la valeur : parfait. Pour une référence, c'est l'**adresse** : `==` répond à la question « est-ce le **même objet** en mémoire ? ».
- `equals` compare le **contenu** de deux objets : « ces deux chaînes contiennent-elles les mêmes caractères ? ».

```
// Fichier : StringEquality.java
void main() {
    String code = "PROMO10";
    String typed = "promo10".toUpperCase();   // crée un NOUVEL objet String

    IO.println(typed);                        // PROMO10
    IO.println(typed == code);                // false : deux objets
différents
    IO.println(typed.equals(code));           // true  : même contenu
    IO.println("promo10".equalsIgnoreCase(code)); // true : ignore
majuscules/minuscules

    String compileTime = "PROMO" + "10";      // calculé par le compilateur
    IO.println(compileTime == code);          // true... par hasard !
}
```

La dernière ligne explique pourquoi ce bug est sournois. Le compilateur réutilise le même objet pour les chaînes **connues à la compilation**. `==` « marche » donc dans vos essais avec des littéraux, puis échoue en production avec une vraie saisie.

**Piège.**

❌

```
if (typed == code) { … }            // compare des adresses
```

✅

```
if ("PROMO10".equals(typed)) { … }  // compare le contenu ; ne plante pas si typed vaut null
```

Placer la constante à gauche (`"PROMO10".equals(typed)`) évite la `NullPointerException` si `typed` vaut `null`. Une autre option sûre est `Objects.equals(a, b)`.

> Règle : `==` pour les primitifs (`int`, `char`, `boolean`…), `equals` pour les objets. En séance 2, vous écrirez votre propre `equals` pour vos classes.

## 2.6 🔒 Débordement d'entier

**Pourquoi ?** Un `int` occupe 32 bits : sa valeur maximale est 2 147 483 647. Que se passe-t-il au-delà ? Java ne signale **rien**. Le résultat « fait le tour » et devient négatif. C'est la faiblesse **CWE-190** (*Integer Overflow or Wraparound*), à l'origine de failles réelles : paniers dont le total devient négatif, contrôles de taille contournés, allocations mémoire trop petites.

**Exemple concret.** Un client malveillant commande 2 millions de pizzas à 12,50 € :

```
// Fichier : Overflow.java
void main() {
    int priceCents = 1250;
    int quantity = 2_000_000;
    int total = priceCents * quantity;       // 2 500 000 000 > 2 147 483 647
    IO.println(total);                        // -1794967296 : un total NÉGATIF

    IO.println(Integer.MAX_VALUE + 1);        // -2147483648
}
```

Si le système de paiement accepte ce montant, la plateforme **rembourse** le client au lieu de le débiter.

**Comment ?** Deux défenses complémentaires :

1. **Valider les entrées** (§4.3) : une quantité au-delà de 20 est refusée avant tout calcul. C'est la première ligne de défense.
2. **Détecter le débordement** : les méthodes `Math.addExact`, `Math.subtractExact` et `Math.multiplyExact` font le calcul et lèvent une `ArithmeticException` si le résultat ne tient pas dans le type.

```
// Fichier : OverflowExact.java (exception attendue)
void main() {
    int priceCents = 1250;
    int feeCents = 399;

    int ok = Math.addExact(Math.multiplyExact(priceCents, 2), feeCents);
    IO.println("Total normal : " + ok);       // 2899

    int quantity = 2_000_000;
    int total = Math.multiplyExact(priceCents, quantity);  //
ArithmeticException: integer overflow
    IO.println("Jamais affiché : " + total);
}
```

**Piège.**

❌ Le calcul déborde en silence :

```
int total = price * quantity + fee;
```

✅ Un débordement arrête le programme au lieu de produire un montant faux :

```
int total = Math.addExact(Math.multiplyExact(price, quantity), fee);
```

❌ Passer à `long` « pour être tranquille » repousse le problème sans le supprimer. Si le calcul est fait en `int` avant d'être stocké, il déborde quand même : `long total = price * quantity;` multiplie deux `int`.

> 🔒 **Sécurité** — Un arrêt franc vaut mieux qu'un résultat faux. Pour intercepter l'`ArithmeticException` et répondre proprement au lieu de s'arrêter, voyez le `try / catch` minimal du §4.6 (détails en séance 3). Retenez dès maintenant : tout calcul sur un montant ou une taille issus d'une saisie doit être **borné** (validation) et **contrôlé** (`*Exact`). Référence : SEI CERT, règle NUM00-J (*Detect or prevent integer overflow*).

## 2.7 🔵 Conversions et constantes

**Conversions.** Java convertit automatiquement vers un type plus large ( `int` → `long` → `double` ). Dans l'autre sens, il faut un **cast** explicite, et l'information perdue l'est en silence :

```
// Fichier : Casts.java
void main() {
    int cents = 1999;
    double euros = cents / 100.0;        // 19.99 : 100.0 est un double
    IO.println(euros);
    IO.println((int) 3.99);              // 3 : troncature, pas d'arrondi
    IO.println((byte) 200);              // -56 : 200 ne tient pas dans un byte
    long big = 3_000_000_000L;
    IO.println((int) big);               // -1294967296 : même problème que le
débordement
    IO.println(Math.round(3.6));         // 4 : arrondi
}
```

**Constantes.** Le mot-clé `final` interdit de réaffecter une variable. Pour une constante de classe, on écrit `static final` et un nom en `SCREAMING_SNAKE_CASE` :

```
// Fichier : Constants.java
static final int MAX_QUANTITY = 20;
static final int FREE_DELIVERY_THRESHOLD_CENTS = 3000;

void main() {
    final int fee = 199;
    // fee = 0;            // ne compile pas : fee est final
    IO.println("Maximum : " + MAX_QUANTITY + ", livraison offerte dès "
            + FREE_DELIVERY_THRESHOLD_CENTS / 100 + " €, frais : " + fee);
}
```

Une constante nommée vaut mieux qu'un « nombre magique » : `quantity <= MAX_QUANTITY` se lit et se modifie en un seul endroit.

## 2.8 🛠️ À vous

> ⏱️ 20 à 30 min, sur votre machine.

**Exercice 1 — Prédire, puis vérifier** (⏱️ 8 min)

Écrivez d'abord sur papier ce qu'affiche chaque ligne. Lancez ensuite le programme.

```java
// Fichier : Predict.java
void main() {
    IO.println(7 / 2);
    IO.println(7 / 2.0);
    IO.println(7 % 3);
    IO.println("Total : " + 1 + 2);
    IO.println(1 + 2 + " €");
    IO.println(Integer.MAX_VALUE + 1);
    IO.println(0.1 + 0.2);
    IO.println("PROMO" == new String("PROMO"));
    IO.println("PROMO".equals(new String("PROMO")));
}
```

▶ Solution de l'exercice 1

**Exercice 2 — Une ligne de ticket** (⏱️ 10 min)

Dans un *compact source file* `TicketLine.java`, déclarez trois variables : le nom du plat (`"Margherita"`), son prix unitaire **en centimes** (`1250`) et la quantité (`2`). Affichez :

```
2 x Margherita = 25,00 €
```

Écrivez pour cela une méthode `static String formatEuros(int cents)` qui transforme `2500` en `"25,00 €"` et `2505` en `"25,05 €"`.

**Indice.** `cents / 100` donne les euros, `cents % 100` les centimes. Attention à `5` centimes, qui doit s'afficher `05`.

▶ Solution de l'exercice 2

**Exercice 3 — Voir un débordement au débogueur** (⏱️ 7 min)

Copiez ce programme dans IntelliJ. Posez un point d'arrêt sur la ligne `orders = orders + 1;`, lancez en *Debug* (§0.4), puis appuyez sur *Resume* plusieurs fois en regardant la variable `orders`. À quel tour devient-elle négative ?

```java
// Fichier : CounterOverflow.java
public class CounterOverflow {
    public static void main(String[] args) {
        int orders = Integer.MAX_VALUE - 2;
        for (int round = 1; round <= 5; round++) {
            orders = orders + 1;
            System.out.println("Tour " + round + " : " + orders);
        }
    }
}
```

Remplacez ensuite `orders + 1` par `Math.addExact(orders, 1)` et relancez. Que se passe-t-il ?

▶ Solution de l'exercice 3

---

# 3. Conditions, boucles, tableaux, méthodes

## 3.1 🟢 `if` / `else`

**Pourquoi ?** Un programme doit prendre des décisions : la livraison est-elle offerte ? la quantité est-elle acceptable ?

**Comment ?** `if (condition) { … } else { … }`. La condition est une expression `boolean`. Les branches `else if` sont testées dans l'ordre : la **première** vraie gagne.

```java
// Fichier : DeliveryFeeIf.java
static int deliveryFeeCents(int subtotalCents) {
    int feeCents;
    if (subtotalCents >= 3000) {
        feeCents = 0;                  // livraison offerte dès 30 €
    } else if (subtotalCents >= 1500) {
        feeCents = 199;
    } else {
        feeCents = 399;
    }
    return feeCents;
}

void main() {
    IO.println(deliveryFeeCents(2350));   // 199
    IO.println(deliveryFeeCents(3000));   // 0
    IO.println(deliveryFeeCents(800));    // 399
}
```

Le compilateur vérifie qu'une variable est affectée avant d'être lue. Sans le dernier `else`, `feeCents` pourrait ne jamais recevoir de valeur : le code ne compile plus (`variable feeCents might not have been initialized`).

**L'opérateur ternaire** `condition ? valeurSiVrai : valeurSiFaux` est un `if`/`else` en une expression. Réservez-le aux cas simples :

```java
String label = feeCents == 0 ? "Livraison offerte" : "Livraison payante";
```

**Piège : oublier les accolades.**

❌ Sans accolades, seule la **première** instruction dépend du `if` ; l'indentation trompe le lecteur :

```java
if (subtotalCents >= 3000)
    feeCents = 0;
    IO.println("Livraison offerte !");   // TOUJOURS affiché
```

✅ Accolades systématiques, même pour une seule ligne.

## 3.2 🟢 `switch` en expression

**Pourquoi ?** Une cascade de `if (zone.equals("CENTRE")) … else if (zone.equals("PERIPHERIE")) …` devient vite illisible quand on choisit parmi plusieurs

valeurs.

**Comment ?** Depuis Java 14, `switch` peut être une **expression** qui produit une valeur. Avec la flèche `->`, chaque `case` est indépendant : pas de `break`.

```
// Fichier : DeliveryFeeSwitch.java
static int deliveryFeeCents(String zone) {
    return switch (zone) {
        case "CENTRE" -> 199;
        case "PERIPHERIE", "BANLIEUE" -> 399;     // plusieurs valeurs
        default -> 599;                            // toutes les autres zones
    };
}

static String dayLabel(int dayOfWeek) {
    return switch (dayOfWeek) {
        case 1, 2, 3, 4, 5 -> "Semaine";
        case 6, 7 -> "Week-end";
        default -> {
            String message = "Jour invalide : " + dayOfWeek;
            yield message;                         // yield : valeur d'un bloc
        }
    };
}

void main() {
    IO.println(deliveryFeeCents("PERIPHERIE"));   // 399
    IO.println(deliveryFeeCents("AEROPORT"));     // 599
    IO.println(dayLabel(6));                      // Week-end
    IO.println(dayLabel(9));                      // Jour invalide : 9
}
```

Règles :

- Un `switch` en expression doit couvrir **tous** les cas possibles. Pour une `String` ou un `int`, il faut donc un `default`. En séance 2, avec un `enum` (`OrderStatus`), le compilateur vérifiera qu'aucune valeur n'est oubliée, sans `default`.
- Un `case` qui a besoin de plusieurs instructions utilise un bloc `{ … }` et renvoie sa valeur avec **`yield`**.
- `switch` sur une `String` qui vaut `null` lève une `NullPointerException`.

**Piège : l'ancien `switch`.**

❌ Vous verrez cette forme dans du code ancien. Un `break` oublié fait « tomber » l'exécution dans le `case` suivant :

```java
int fee = 0;
switch (zone) {
    case "CENTRE":
        fee = 199;            // pas de break : on continue !
    case "PERIPHERIE":
        fee = 399;
        break;
    default:
        fee = 599;
}
// zone = "CENTRE" donne fee = 399
```

✅ Utilisez la forme flèche, en expression, comme dans `DeliveryFeeSwitch.java`.

## 3.3 🟢 Boucles `for`, `while` et for-each

**Pourquoi ?** Répéter une action : parcourir les lignes d'une commande, compter des plats, réessayer jusqu'à une condition.

### Comment ?

| Boucle | Quand l'utiliser |
|---|---|
| `for (init; condition; pas)` | on connaît le nombre de tours |
| `for (Type x : tableau)` (for-each) | parcourir tous les éléments, sans besoin de l'indice |
| `while (condition)` | répéter tant qu'une condition est vraie |
| `do { … } while (condition);` | idem, mais au moins un tour (rare) |

```java
// Fichier : Loops.java
void main() {
    // for : 3 tours, table vaut 1, 2, puis 3
    for (int table = 1; table <= 3; table++) {
        IO.println("Table " + table + " servie");
    }

    // while : combien de pizzas à 12,50 € avec 30 € ?
    int budgetCents = 3000;
    int pizzas = 0;
    while (budgetCents >= 1250) {
        budgetCents -= 1250;
        pizzas++;
    }
    IO.println(pizzas + " pizzas, reste " + budgetCents + " centimes");  // 2
pizzas, reste 500

    // break : arrêter la boucle ; continue : passer au tour suivant
    int[] quantities = {2, 0, 3, -1, 4};
    for (int q : quantities) {
        if (q == 0) {
            continue;                   // ligne vide : on l'ignore
        }
        if (q < 0) {
            IO.println("Quantité invalide, arrêt");
            break;                      // on sort de la boucle
        }
        IO.println("Quantité : " + q);
    }
}
```

**Piège : la boucle infinie.**

❌ La condition ne devient jamais fausse : le programme ne s'arrête pas.

```java
int budgetCents = 3000;
while (budgetCents >= 1250) {
    IO.println("Encore une pizza");   // budgetCents ne diminue jamais
}
```

✅ Vérifiez que **chaque tour** rapproche la boucle de sa fin (ici, `budgetCents -= 1250;`).

## 3.4 🟢 Tableaux

**Pourquoi ?** Une commande contient plusieurs plats. Créer `price1`, `price2`, `price3`… ne passe pas à l'échelle.

**Comment ?** Un **tableau** stocke un nombre **fixe** de valeurs du même type, accessibles par un **indice** qui commence à 0.

```
int[] pricesCents = {1250, 990, 350};   // 3 éléments : indices 0, 1, 2
int[] quantities = new int[3];          // 3 éléments valant 0
quantities[0] = 2;                      // écriture
int first = pricesCents[0];             // lecture : 1250
int size = pricesCents.length;          // 3 (attribut, sans parenthèses)
```

**Exemple concret : le total d'une commande.** Deux tableaux « parallèles » : l'élément `i` de `quantities` correspond au plat `i` de `pricesCents`.

```
// Fichier : OrderTotal.java
void main() {
    String[] dishes = {"Margherita", "Salade César", "Limonade"};
    int[] pricesCents = {1250, 990, 350};
    int[] quantities = {2, 1, 3};

    int totalCents = 0;
    for (int i = 0; i < pricesCents.length; i++) {
        int lineCents = pricesCents[i] * quantities[i];
        IO.println(dishes[i] + " x" + quantities[i] + " = " + lineCents);
        totalCents += lineCents;
    }
    IO.println("Total : " + totalCents + " centimes");   // 4540

    int itemCount = 0;
    for (int q : quantities) {              // for-each : pas besoin d'indice
        itemCount += q;
    }
    IO.println(itemCount + " articles");                // 6 articles

    IO.println(Arrays.toString(pricesCents));           // [1250, 990, 350]
}
```

**Piège 1 : sortir du tableau.**

❌ `i <= length` fait un tour de trop. L'indice 3 n'existe pas dans un tableau de 3 éléments :

```java
// Fichier : OutOfBounds.java (exception attendue)
void main() {
    int[] pricesCents = {1250, 990, 350};
    for (int i = 0; i <= pricesCents.length; i++) {   // <= au lieu de <
        IO.println(pricesCents[i]);                    //
ArrayIndexOutOfBoundsException au 4e tour
    }
}
```

✅ `i < pricesCents.length`, ou un for-each quand l'indice est inutile. Notez que Java **vérifie** chaque accès : en C, cette erreur lirait silencieusement la mémoire voisine.

**Piège 2 : afficher un tableau.**

❌ `IO.println(pricesCents)` affiche quelque chose comme `[I@56a6d5a6` (le type et un identifiant interne), pas le contenu.

✅ `IO.println(Arrays.toString(pricesCents))` affiche `[1250, 990, 350]`.

> Un tableau a une taille **fixe**. Pour une commande dont le nombre de lignes varie, on utilisera une `List` en séance 2.

## 3.5 🟢 Méthodes `static`

**Pourquoi ?** Copier-coller la boucle du total à chaque endroit où on en a besoin, c'est dupliquer les bugs. Une **méthode** donne un **nom** à un calcul, le rend **réutilisable** et surtout **testable** (§5).

**Comment ?** La **signature** d'une méthode décrit ce qu'elle attend et ce qu'elle renvoie :

```
static   int        totalCents  (int[] prices, int[] quantities)
  |       |            |                     |
  |       |            |                     └─ paramètres : type et nom de
chaque entrée
  |       |            └─ nom, en lowerCamelCase, qui décrit l'intention
  |       └─ type de la valeur renvoyée (void si rien)
  └─ méthode de classe : appelable sans objet
```

- `return valeur;` termine la méthode et renvoie la valeur à l'appelant.
- Une méthode `void` ne renvoie rien ; `return;` (sans valeur) permet d'en sortir plus tôt.

- Les **paramètres** sont des variables locales initialisées avec les **arguments** passés à l'appel.

**Exemple concret : une classe `OrderCalculator`.** Nous la réutiliserons dans les tests (§5.5). C'est une classe nommée, avec un `main` classique pour l'essayer.

```java
// Fichier : OrderCalculator.java
public class OrderCalculator {

    /** Total d'une commande, en centimes. Lève ArithmeticException en cas de débordement. */
    public static int totalCents(int[] unitPricesCents, int[] quantities) {
        if (unitPricesCents.length != quantities.length) {
            throw new IllegalArgumentException("Tableaux de tailles différentes");
        }
        int total = 0;
        for (int i = 0; i < unitPricesCents.length; i++) {
            int line = Math.multiplyExact(unitPricesCents[i], quantities[i]);
            total = Math.addExact(total, line);
        }
        return total;
    }

    /** Applique une remise en pourcentage (0 à 100). Les fractions de centime sont tronquées. */
    public static int applyDiscount(int totalCents, int percent) {
        if (percent < 0 || percent > 100) {
            throw new IllegalArgumentException("Remise invalide : " + percent);
        }
        int discount = Math.multiplyExact(totalCents, percent) / 100;
        return totalCents - discount;
    }

    /** Frais de livraison : offerts dès 30 €, sinon selon la zone. */
    public static int deliveryFeeCents(int subtotalCents, String zone) {
        if (subtotalCents >= 3000) {
            return 0;
        }
        return switch (zone) {
            case "CENTRE" -> 199;
            case "PERIPHERIE", "BANLIEUE" -> 399;
            default -> 599;
        };
    }

    public static void main(String[] args) {
        int[] prices = {1250, 350};
        int[] quantities = {2, 1};
        int subtotal = totalCents(prices, quantities);                  // 2850
        int discounted = applyDiscount(subtotal, 10);                   // 2565
        int toPay = Math.addExact(discounted, deliveryFeeCents(discounted, "CENTRE"));
        IO.println("Sous-total : " + subtotal);
        IO.println("Après remise : " + discounted);
```

```
        IO.println("À payer : " + toPay);                    // 2764
        IO.println(applyDiscount(1999, 15));                 // 1700
    }
}
```

Quelques remarques :

- `applyDiscount` **vérifie ses paramètres** avant de calculer. Une remise de 150 % donnerait un total négatif. `throw new IllegalArgumentException(...)` arrête la méthode avec un message explicite ; nous étudierons les exceptions en séance 3.
- `applyDiscount(1999, 15)` : 15 % de 1999 font 299,85 centimes. La division entière **tronque** à 299, le client paie 1700. Qui doit gagner le centime ? C'est une règle métier. Avec `BigDecimal` (séance 2), vous choisirez explicitement le mode d'arrondi.
- Les commentaires `/** … */` sont des commentaires **Javadoc** : ils documentent la méthode et apparaissent dans votre IDE.

**Piège : oublier d'utiliser la valeur renvoyée.**

❌ La méthode calcule un résultat… que personne ne récupère :

```
int total = 2850;
OrderCalculator.applyDiscount(total, 10);   // résultat perdu
IO.println(total);                          // 2850 : rien n'a changé
```

✅ Une méthode ne modifie pas les variables `int` de l'appelant. Récupérez le résultat :

```
int total = 2850;
total = OrderCalculator.applyDiscount(total, 10);
IO.println(total);                          // 2565
```

🔵 **Passage par valeur.** Java passe **toujours** une copie de l'argument. Pour un `int`, c'est une copie de la valeur : la méthode ne peut pas modifier la variable de l'appelant. Pour un tableau ou un objet, c'est une copie de la **référence** : la méthode ne peut pas remplacer le tableau de l'appelant, mais elle peut **modifier son contenu**.

```java
// Fichier : PassByValue.java
static void addOne(int value) { value++; }
static void clearFirst(int[] values) { values[0] = 0; }

void main() {
    int quantity = 3;
    addOne(quantity);
    IO.println(quantity);                // 3 : la copie a changé, pas
l'original
    int[] quantities = {3, 4};
    clearFirst(quantities);
    IO.println(quantities[0]);           // 0 : le contenu du tableau a changé
}
```

C'est une source de bugs et de failles : une méthode qui reçoit un tableau peut le modifier dans votre dos. En séance 2, nous verrons les copies défensives.

## 3.6 🔵 Surcharge

Plusieurs méthodes peuvent porter le **même nom** si leurs **paramètres** diffèrent (nombre ou types). C'est la **surcharge** (*overloading*). Le compilateur choisit la bonne version selon les arguments.

```java
// Fichier : Overloading.java
static String formatPrice(int cents) {
    return formatPrice(cents, "€");          // réutilise la version la plus
complète
}

static String formatPrice(int cents, String currency) {
    return String.format("%d,%02d %s", cents / 100, cents % 100, currency);
}

void main() {
    IO.println(formatPrice(1250));           // 12,50 €
    IO.println(formatPrice(990, "EUR"));     // 9,90 EUR
}
```

Le type de retour seul ne suffit pas : deux méthodes `int f(int)` et `long f(int)` ne compilent pas ensemble.

## 3.7 🔵 Portée des variables

La **portée** d'une variable locale va de sa déclaration à la fin du **bloc** `{ … }` qui la contient. Un bloc intérieur voit les variables du bloc extérieur, pas l'inverse.

```
// Fichier : Scope.java
void main() {
    int totalCents = 0;                          // visible dans tout main
    int[] prices = {1250, 350};
    for (int i = 0; i < prices.length; i++) {  // i : visible dans la boucle
seulement
        int line = prices[i];                    // line : idem
        totalCents += line;
    }
    // IO.println(i);     // ne compile pas : i n'existe plus ici
    // IO.println(line);  // ne compile pas
    IO.println(totalCents);                      // 1600
}
```

Déclarez chaque variable **au plus près** de son utilisation, dans la portée la plus petite possible : moins de variables visibles, moins d'erreurs.

## 3.8 🟢 Aperçu : lire une `List` et une `Map`

**Pourquoi ?** Les collections sont au programme de la séance 2. Mais dès aujourd'hui, vous lirez du code qui en utilise : des arguments reçus sous forme de liste, des données rangées par clé. Cette section donne **juste ce qu'il faut pour lire et utiliser** ces deux types. Les explications complètes (interfaces, génériques, `equals` / `hashCode`) viendront en séance 2.

**Une `List` : une suite ordonnée, comme un tableau qui sait grandir.** `List<String>` se lit « liste de `String` ».

| Opération | Exemple | Résultat |
|---|---|---|
| `size()` | `args.size()` | le nombre d'éléments (comme `length` d'un tableau) |
| `get(i)` | `args.get(0)` | l'élément d'indice `i`, de `0` à `size() - 1` |
| `getFirst()` | `args.getFirst()` | le premier élément |
| `isEmpty()` | `args.isEmpty()` | `true` si la liste est vide |

| Opération | Exemple | Résultat |
|---|---|---|
| `subList(debut, fin)` | `args.subList(1, args.size())` | une vue sur une partie (fin exclue) |
| for-each | `for (String a : args) { … }` | parcourt tous les éléments |

**Une `Map` : un dictionnaire qui associe une clé à une valeur.** `Map<String, Integer>` associe des `String` (les clés) à des `Integer` (les valeurs). Une clé est unique : un nouveau `put` sur la même clé **remplace** l'ancienne valeur.

| Opération | Exemple | Résultat |
|---|---|---|
| `put(cle, valeur)` | `stock.put("Margherita", 12)` | ajoute ou remplace |
| `get(cle)` | `stock.get("Margherita")` | la valeur, ou `null` si la clé est absente |
| `getOrDefault(cle, defaut)` | `stock.getOrDefault("Sushi", 0)` | la valeur, ou `defaut` si absente |
| `containsKey(cle)` | `stock.containsKey("Sushi")` | `true` ou `false` |
| `remove(cle)` | `stock.remove("Tiramisu")` | retire la clé ; renvoie l'ancienne valeur ou `null` |
| `size()` | `stock.size()` | le nombre de clés |

**Exemple concret** : une commande arrive sous forme de mots ( `"ADD Margherita 2"` ), et le stock de FoodFast est rangé par nom de plat.

```java
// Fichier : ListMapPreview.java
void main() {
    List<String> words = List.of("ADD", "Margherita", "2");
    IO.println(words.size());          // 3
    IO.println(words.getFirst());      // ADD
    IO.println(words.get(1));          // Margherita
    for (String w : words.subList(1, words.size())) {
        IO.println("- " + w);          // - Margherita, puis - 2
    }

    Map<String, Integer> stock = new HashMap<>();
    stock.put("Margherita", 12);
    stock.put("Tiramisu", 4);
    stock.put("Tiramisu", 3);                       // remplace 4 par 3
    IO.println(stock.get("Tiramisu"));              // 3
    IO.println(stock.get("Sushi"));                 // null : clé absente
    IO.println(stock.getOrDefault("Sushi", 0));     // 0
    IO.println(stock.containsKey("Margherita"));    // true
    IO.println(stock.remove("Tiramisu"));           // 3
    IO.println(stock.size());                       // 1
}
```

**Piège.**

❌ Utiliser directement le résultat de `get` : si la clé est absente, on obtient `null`, et le calcul plante (`NullPointerException`).

```java
int left = stock.get("Sushi") - 1;              // NullPointerException
```

✅ Prévoir l'absence :

```java
int left = stock.getOrDefault("Sushi", 0);      // 0 si absent
```

❌ `List.of(...)` crée une liste **non modifiable** : `words.add("x")` lève une `UnsupportedOperationException`. Pour une liste qu'on remplit, on écrit `new ArrayList<>()` (séance 2).

## 3.9 🛠️ À vous

> ⏱️ 30 à 40 min, sur votre machine. Commencez par les exercices 1 et 2 ; le 3 est plus long.

**Exercice 1 — Le jour de la semaine** (⏱ 8 min)

Écrivez `static String dayLabel(int day)` qui renvoie `"lundi"` pour 1, `"mardi"` pour 2… `"dimanche"` pour 7, et `"jour invalide"` pour tout autre nombre. Utilisez un `switch` en expression (§3.2). Affichez le résultat pour 1, 7, 0 et 8.

▶ Solution de l'exercice 1

**Exercice 2 — Des sommes, et le débogueur** (⏱ 12 min)

1. Écrivez `static long sumUpTo(int n)` : la somme `1 + 2 + … + n`. Pour `n ≤ 0`, elle vaut `0`.
2. Écrivez `static long sumEvenUpTo(int n)` : la somme des nombres **pairs** de `2` à `n`.
3. Posez un point d'arrêt dans la boucle de `sumUpTo`, lancez en *Debug* avec `sumUpTo(4)` et suivez `i` et la somme avec *Step Over* (F8) pendant trois tours.
4. Vérifiez : `sumUpTo(100)` vaut `5050`. Pourquoi le type de retour est-il `long` et pas `int` ? Essayez `sumUpTo(100_000)` avec un `int`.

▶ Solution de l'exercice 2

**Exercice 3 — Statistiques d'un panier** (⏱ 15 min)

Créez une classe nommée `BasketStats` (elle servira pour les tests du §5.8) avec trois méthodes `static` :

- `maxCents(int[] prices)` : le prix le plus élevé ;
- `countAbove(int[] prices, int thresholdCents)` : le nombre de prix **strictement** supérieurs au seuil ;
- `averageCents(int[] prices)` : la moyenne, arrondie à l'entier inférieur.

Avec `{1250, 450, 800, 1590}` : maximum `1590`, `countAbove(…, 1000)` vaut `2`, moyenne `1022`.

Que doivent faire `maxCents` et `averageCents` sur un tableau **vide** ? Décidez, puis programmez-le.

**Indice.** Pour le maximum, partez du premier élément, pas de `0` : que se passerait-il avec des valeurs négatives ?

▶ Solution de l'exercice 3

---

# 4. Chaînes de caractères et validation

## 4.1 🟢 Méthodes de `String`

Une `String` est une suite de caractères `char`, indexés à partir de 0. Elle est **immuable** : aucune méthode ne la modifie. Les méthodes qui « transforment » renvoient une **nouvelle** chaîne.

| Méthode | Exemple | Résultat |
|---|---|---|
| `length()` | `"Pizza".length()` | `5` |
| `charAt(i)` | `"Pizza".charAt(0)` | `'P'` |
| `substring(debut)` / `substring(debut, fin)` | `"order-123".substring(6)` | `"123"` (fin exclue) |
| `indexOf(s)` | `"order-123".indexOf("-")` | `5` (ou `-1` si absent) |
| `contains(s)` | `"Pizza 4 fromages".contains("fromage")` | `true` |
| `startsWith(s)` / `endsWith(s)` | `"order-123".startsWith("order-")` | `true` |
| `toUpperCase()` / `toLowerCase()` | `"promo10".toUpperCase()` | `"PROMO10"` |
| `strip()` | `" 3 ".strip()` | `"3"` (retire les espaces autour) |
| `isEmpty()` / `isBlank()` | `" ".isBlank()` | `true` (vide ou que des espaces) |
| `replace(a, b)` | `"12.50".replace(".", ",")` | `"12,50"` |
| `split(regex)` | `"pizza;2".split(";")` | tableau `{"pizza", "2"}` |
| `equals(s)` / `equalsIgnoreCase(s)` | `"Pizza".equalsIgnoreCase("PIZZA")` | `true` |

| Méthode | Exemple | Résultat |
|---|---|---|
| `String.format(...)` / `formatted(...)` | `String.format("%02d", 5)` | `"05"` |

```java
// Fichier : StringMethods.java
void main() {
    String orderId = "order-123";
    int dash = orderId.indexOf("-");
    String number = orderId.substring(dash + 1);
    IO.println(number);                                  // 123
    IO.println(orderId.startsWith("order-"));            // true

    String line = "Margherita;2";
    String[] parts = line.split(";");
    IO.println(parts[0] + " / " + parts[1]);             // Margherita / 2

    String typed = "  promo10 ";
    String code = typed.strip().toUpperCase();
    IO.println("[" + code + "] " + code.length());       // [PROMO10] 7

    String dish = "Tiramisu";
    for (int i = 0; i < dish.length(); i++) {
        IO.print(dish.charAt(i) + " ");                  // T i r a m i s u
    }
    IO.println();
}
```

### Piège : croire qu'une méthode modifie la chaîne.

❌

```java
String code = "promo10";
code.toUpperCase();          // renvoie "PROMO10"… qui est jeté
IO.println(code);            // promo10
```

✅

```java
String code = "promo10";
code = code.toUpperCase();   // on récupère la nouvelle chaîne
IO.println(code);            // PROMO10
```

## 4.2 🟢 `StringBuilder` : construire un ticket

**Pourquoi ?** Comme une `String` est immuable, `s = s + "..."` dans une boucle crée **une nouvelle chaîne à chaque tour** et recopie tout le texte précédent. Pour un ticket de 3 lignes, cela ne se voit pas. Pour un export de 100 000 commandes, le programme devient très lent.

**Comment ?** Un `StringBuilder` est un tampon de texte **modifiable**. On ajoute des morceaux avec `append` (chaînable), puis on obtient la `String` finale avec `toString()`.

```
// Fichier : Ticket.java
static String formatEuros(int cents) {
    return String.format("%d,%02d €", cents / 100, cents % 100);
}

static String ticket(String[] names, int[] quantities, int[] pricesCents) {
    StringBuilder sb = new StringBuilder();
    sb.append("====== FoodFast ======\n");
    int totalCents = 0;
    for (int i = 0; i < names.length; i++) {
        int lineCents = Math.multiplyExact(quantities[i], pricesCents[i]);
        totalCents = Math.addExact(totalCents, lineCents);
        sb.append(String.format("%-12s x%d %10s%n", names[i], quantities[i],
formatEuros(lineCents)));
    }
    sb.append("----------------------\n");
    sb.append(String.format("%-15s %10s%n", "TOTAL", formatEuros(totalCents)));
    return sb.toString();
}

void main() {
    String[] names = {"Margherita", "Tiramisu", "Limonade"};
    int[] quantities = {2, 1, 3};
    int[] pricesCents = {1250, 650, 350};
    IO.print(ticket(names, quantities, pricesCents));
}
```

Sortie :

```
====== FoodFast ======
Margherita   x2    25,00 €
Tiramisu     x1     6,50 €
Limonade     x3    10,50 €
----------------------
TOTAL              42,00 €
```

Les codes de `String.format` les plus utiles : `%s` (chaîne), `%d` (entier), `%02d` (entier sur 2 chiffres complété par des zéros), `%-12s` (chaîne alignée à gauche sur 12 caractères), `%10s` (alignée à droite), `%n` (saut de ligne).

**Piège.**

❌ Concaténation dans une boucle :

```
String report = "";
for (int i = 0; i < 100_000; i++) {
    report = report + "order-" + i + "\n";   // 100 000 chaînes recréées et
recopiées
}
```

✅

```
StringBuilder report = new StringBuilder();
for (int i = 0; i < 100_000; i++) {
    report.append("order-").append(i).append('\n');
}
String result = report.toString();
```

En dehors des boucles, `+` reste parfaitement lisible et efficace : le compilateur l'optimise.

## 4.3 🔒 Valider les entrées

**Pourquoi ?** Toute donnée qui vient de l'extérieur (formulaire, URL, fichier, API, ligne de commande) peut être **fausse**, par erreur ou par malveillance. Dans FoodFast, une quantité saisie arrive sous forme de **texte**. Voici ce qu'un utilisateur peut envoyer : `"3"`, `" 3"`, `""`, `"abc"`, `"-2"`, `"0"`, `"3.5"`, `"999999999999"`, ou `"٣"` (le chiffre 3 en écriture arabe-indienne). Sans validation, on obtient au mieux un plantage, au pire un total négatif (§2.6). La validation absente ou incomplète est la racine de nombreuses failles : CWE-20 (*Improper Input Validation*), injections, débordements.

**Comment ?** On vérifie dans cet ordre :

1. **Présence** : la valeur n'est ni `null`, ni vide.
2. **Format** : uniquement les caractères autorisés, avec une longueur maximale.
3. **Bornes** : la valeur a un sens métier (une quantité entre 1 et 20).
4. Seulement **ensuite**, on convertit et on calcule.

On décrit ce qui est **autorisé** (liste blanche, *allowlist*), plutôt que d'essayer d'énumérer tout ce qui est interdit (liste noire) : on oublie toujours un cas.

```java
// Fichier : InputValidator.java
public class InputValidator {

    public static final int MAX_QUANTITY = 20;

    /** Vrai si input représente une quantité entière entre 1 et MAX_QUANTITY,
en chiffres ASCII. */
    public static boolean isValidQuantity(String input) {
        if (input == null || input.isBlank() || input.length() > 2) {
            return false;                       // présence et longueur
        }
        for (char c : input.toCharArray()) {
            if (c < '0' || c > '9') {
                return false;                   // format : chiffres 0 à 9
uniquement
            }
        }
        int quantity = Integer.parseInt(input); // sûr : 1 ou 2 chiffres ASCII
        return quantity >= 1 && quantity <= MAX_QUANTITY;   // bornes
    }

    /** Vrai si code ressemble à un code promo : 4 à 12 lettres majuscules ou
chiffres. */
    public static boolean isValidPromoCode(String code) {
        if (code == null || code.length() < 4 || code.length() > 12) {
            return false;
        }
        for (char c : code.toCharArray()) {
            boolean upper = c >= 'A' && c <= 'Z';
            boolean digit = c >= '0' && c <= '9';
            if (!upper && !digit) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        String[] inputs = {"3", "20", "21", "0", " 3", "", "abc", "-2", "3.5",
"999999999999", "٢", null};
        for (String input : inputs) {
            IO.println("[" + input + "] -> " + isValidQuantity(input));
        }
        IO.println(isValidPromoCode("PROMO10"));       // true
        IO.println(isValidPromoCode("PROMO10'--"));    // false
    }
}
```

Points importants :

- **La longueur est vérifiée en premier.** Elle protège tout le reste : `Integer.parseInt("999999999999")` lèverait une `NumberFormatException` (le nombre ne tient pas dans un `int`), et une chaîne de 10 millions de caractères ferait travailler la boucle pour rien.
- **On teste `'0'` à `'9'` explicitement**, et non `Character.isDigit(c)`. Pourquoi ? `Character.isDigit` accepte **tous** les chiffres Unicode (arabes-indiens, devanagari…), et `Integer.parseInt` aussi :

```
// Fichier : UnicodeDigits.java
void main() {
    String arabicThree = "٣ //                          ;"٣
    IO.println(Character.isDigit(arabicThree.charAt(0)));  // true
    IO.println(Integer.parseInt(arabicThree));             // 3
}
```

Un filtre qui laisse passer des caractères inattendus peut contourner un contrôle situé plus loin, par exemple un autre système qui ne lit pas ces chiffres de la même façon. La liste blanche ASCII élimine le problème.

- `" 3"` est refusé : l'espace n'est pas un chiffre. Si votre règle métier tolère les espaces autour, faites `input.strip()` **avant** d'appeler la validation, de façon explicite.

**Piège.**

❌ Convertir d'abord, valider ensuite (ou jamais) :

```
int quantity = Integer.parseInt(input);       // plante sur "abc", accepte "٣"
et "-2"
int total = quantity * priceCents;            // déborde avec une grande valeur
```

✅ Valider, puis convertir, puis calculer avec des opérations contrôlées :

```
if (!InputValidator.isValidQuantity(input)) {
    IO.println("Quantité invalide (1 à " + InputValidator.MAX_QUANTITY + ")");
    return;
}
int quantity = Integer.parseInt(input);
int total = Math.multiplyExact(quantity, priceCents);
```

> 🔒 **Sécurité** — Valider tôt, au plus près de l'entrée, et refuser par défaut. Le message d'erreur affiché à l'utilisateur dit **ce qui est attendu** (« 1 à 20 »), sans détail technique (pas de pile d'appels, pas de nom de classe) : nous y reviendrons en séance 3. Référence : OWASP Input Validation Cheat Sheet.

**Lire une saisie au clavier.** `IO.readln("Quantité ? ")` affiche une invite et renvoie la ligne tapée (une `String`). Les exemples vérifiés de ce support n'utilisent pas le clavier, mais vous pouvez essayer ce programme chez vous :

```
void main() {
    String input = IO.readln("Quantité de Margherita ? ");
    if (!InputValidator.isValidQuantity(input)) {
        IO.println("Quantité invalide : entrez un nombre entre 1 et 20.");
        return;
    }
    int quantity = Integer.parseInt(input);
    IO.println("Total : " + Math.multiplyExact(quantity, 1250) + " centimes");
}
```

(Placez-le dans le même dossier que `InputValidator.java`, puis lancez `java Saisie.java` : depuis Java 22, le lanceur compile aussi les autres fichiers du dossier dont il a besoin.)

## 4.4 🔵 Text blocks

Un **text block** est une chaîne sur plusieurs lignes, délimitée par trois guillemets `"""`. Il est pratique pour un gabarit de ticket, un message long, du JSON ou une requête SQL (séance 4).

```
// Fichier : TextBlocks.java
void main() {
    String header = """
            ====== FoodFast ======
            Commande : %s
            Client   : %s
            """.formatted("order-123", "Alice");
    IO.print(header);

    String json = """
        {"dish": "Margherita", "quantity": 2}""";
    IO.println(json);
}
```

Règles :

- Le `"""` d'ouverture est **seul sur sa ligne** : le texte commence à la ligne suivante.
- L'indentation commune à toutes les lignes (y compris celle du `"""` fermant) est **retirée**. Placez le `"""` fermant sous le texte, au même niveau, pour contrôler cette indentation.
- Si le `"""` fermant est sur sa propre ligne, la chaîne se termine par un saut de ligne ; s'il suit le dernier caractère (comme pour `json`), non.
- `.formatted(...)` fonctionne comme `String.format`.

> 🔒 **Sécurité** — Un text block reste une simple chaîne. Y insérer une saisie utilisateur par `formatted` ou `+` pour fabriquer du SQL ou du HTML ouvre la porte aux **injections**. En séance 4, nous utiliserons `PreparedStatement` pour le SQL.

## 4.5 🔵 `import module` (JEP 511)

Dans une classe nommée, chaque classe hors de `java.lang` doit être importée : `import java.util.List;`, `import java.util.Map;` … Depuis Java 25, `import module` importe d'un coup toutes les classes publiques d'un **module** :

```
// Fichier : ModuleImport.java
import module java.base;      // List, Map, Arrays, LocalDate, BigDecimal…

public class ModuleImport {
    public static void main(String[] args) {
        List<String> dishes = List.of("Margherita", "Tiramisu");
        int[] prices = {1250, 650};
        IO.println(dishes + " " + Arrays.toString(prices));
        IO.println(LocalDate.of(2026, 9, 1).getDayOfWeek());   // TUESDAY
    }
}
```

C'est exactement ce que fait implicitement un *compact source file*. `List` et `LocalDate` seront présentés en séance 2.

Limite : si deux modules importés contiennent une classe de même nom, le nom devient **ambigu** et il faut un import précis pour trancher. Exemple : `java.base` et `java.desktop` contiennent tous deux une classe `List`. Dans un vrai projet, beaucoup d'équipes préfèrent encore les imports classes par classes, que l'IDE gère automatiquement.

## 4.6 🟢 Convertir un texte en nombre : `Long.parseLong` et ses exceptions

**Pourquoi ?** Une donnée reçue de l'extérieur (formulaire, fichier, réseau) est **toujours du texte**. Pour calculer, il faut la convertir. Et la conversion peut échouer.

**Comment ?** `Integer.parseInt(s)` et `Long.parseLong(s)` convertissent une `String` en `int` ou en `long`. Si le texte n'est pas un nombre valide, ou s'il ne tient pas dans le type, elles lèvent une `NumberFormatException`.

Jusqu'ici, une exception arrêtait le programme. On peut aussi l'**intercepter** avec `try` / `catch` : si une instruction du bloc `try` lève l'exception nommée dans le `catch`, Java saute directement dans le `catch`. Voici le strict minimum ; les exceptions sont détaillées en séance 3.

**Exemple concret** : ajouter une quantité au compteur de commandes d'un plat, sans planter sur une saisie invalide ni sur un débordement (§2.6).

```
// Fichier : ParseAndAdd.java
/** Renvoie un message décrivant le résultat de current + text. */
static String addQuantity(long current, String text) {
    try {
        long delta = Long.parseLong(text);            // NumberFormatException
?
        long total = Math.addExact(current, delta);   // ArithmeticException ?
        return "Nouveau total : " + total;
    } catch (NumberFormatException e) {
        return "Erreur : ce n'est pas un nombre entier";
    } catch (ArithmeticException e) {
        return "Erreur : dépassement de capacité";
    }
}

void main() {
    IO.println(addQuantity(10, "5"));                   // Nouveau total : 15
    IO.println(addQuantity(10, "abc"));                 // Erreur : ce n'est
pas un nombre entier
    IO.println(addQuantity(10, "3.5"));                 // Erreur : ce n'est
pas un nombre entier
    IO.println(addQuantity(10, "99999999999999999999")); // Erreur : ce n'est
pas un nombre entier (trop grand pour un long)
    IO.println(addQuantity(Long.MAX_VALUE, "1"));       // Erreur :
dépassement de capacité
}
```

Le programme ne s'arrête plus : chaque cas d'erreur produit une réponse claire.

**Piège : `parseLong` est plus permissif qu'on ne le croit.**

```
// Fichier : ParseLenient.java
void main() {
    IO.println(Long.parseLong("+5"));    // 5   : le signe + est accepté
    IO.println(Long.parseLong("007"));   // 7   : les zéros en tête aussi
    IO.println(Long.parseLong("-0"));    // 0
    IO.println(Long.parseLong("٣"));     // 3   : chiffre arabe-indien (§4.3)
}
```

En revanche `" 5"` (avec une espace) est refusé. Si votre spécification est **plus stricte** que `parseLong` (« uniquement des chiffres ASCII, sans zéro en tête »), c'est à vous de valider le format **avant** de convertir, comme en §4.3.

❌ Laisser l'exception remonter jusqu'à arrêter tout le programme, alors que l'erreur vient d'une simple saisie.

✅ Intercepter **précisément** l'exception attendue (`NumberFormatException`, pas « toutes les erreurs ») et répondre par un message clair.

> 🔒 **Sécurité** — Un programme qui s'arrête sur une entrée invalide est un programme qu'un attaquant peut **faire tomber** à volonté : c'est un déni de service. Toute conversion d'une donnée extérieure doit prévoir l'échec.

## 4.7 🔵 Caractères et octets : UTF-8

**Pourquoi ?** Pour Java, une `String` est une suite de **caractères**. Mais un fichier ou une connexion réseau transportent des **octets** (`byte`). Le passage de l'un à l'autre s'appelle l'**encodage**, et aujourd'hui l'encodage standard est **UTF-8**. En UTF-8, un caractère occupe **de 1 à 4 octets** : 1 pour les lettres ASCII (`a`, `Z`, `7`), 2 pour `é` ou `ç`, 3 pour `€`, 4 pour un emoji.

Conséquence : **le nombre de caractères et le nombre d'octets diffèrent** dès qu'il y a un accent.

```java
// Fichier : Utf8Bytes.java
import java.nio.charset.StandardCharsets;

void main() {
    String dish = "Crème brûlée";
    byte[] bytes = dish.getBytes(StandardCharsets.UTF_8);
    IO.println(dish.length());          // 12 caractères
    IO.println(bytes.length);           // 15 octets : è, û et é valent 2
octets chacun

    IO.println("é".length());                                    // 1
    IO.println("é".getBytes(StandardCharsets.UTF_8).length);    // 2
    IO.println("€".getBytes(StandardCharsets.UTF_8).length);    // 3

    String back = new String(bytes, StandardCharsets.UTF_8);    // octets ->
texte
    IO.println(back.equals(dish));      // true
}
```

**Piège.**

❌ Annoncer une taille en caractères quand le destinataire attend des octets. Beaucoup de protocoles réseau et de formats de fichier préfixent une donnée par sa **longueur en octets**. Avec `length()`, le message est faux dès le premier accent.

❌ Oublier le charset : `getBytes()` et `new String(bytes)` sans argument utilisent l'encodage par défaut de la machine. Depuis Java 18, c'est UTF-8 partout, mais écrire `StandardCharsets.UTF_8` rend le code explicite.

✅ Convertir explicitement avec `StandardCharsets.UTF_8`, et compter les octets sur le tableau obtenu.

## 4.8 🛠️ À vous

> ⏱️ 25 à 35 min, sur votre machine.

**Exercice 1 — Nettoyer un code promo saisi** (⏱️ 8 min)

Les clients tapent leur code promo n'importe comment : `" promo10 "`, `"Promo10"`, `"PROMO10"`. Écrivez `static String normalizePromoCode(String raw)` qui retire les espaces au début et à la fin, puis met en majuscules. Si `raw` vaut `null`, la méthode renvoie `""`.

Vérifiez ensuite, avec `InputValidator.isValidPromoCode` (§4.3), que les trois saisies ci-dessus deviennent valides, mais pas `"PROMO 10"` ni `"PROMO10'--"`.

▶ Solution de l'exercice 1

**Exercice 2 — Convertir avec `try` / `catch` … et découvrir un piège** (⏱️ 10 min)

Écrivez `static int parseQuantity(String text)` qui renvoie la quantité (entre 1 et 20) ou `-1` si la saisie est invalide. Utilisez `Integer.parseInt` dans un `try` / `catch` (§4.6), puis vérifiez les bornes.

Testez avec : `"3"`, `" 3"`, `"abc"`, `"0"`, `"21"`, `"99999999999"`, `null`, et `"٣"` (copiez-le tel quel : c'est le chiffre 3 en écriture arabe-indienne). Comparez avec `InputValidator.isValidQuantity` : laquelle des deux méthodes accepte `"٣"` ? Pourquoi est-ce un problème ?

▶ Solution de l'exercice 2

**Exercice 3 — Un ticket complet** (⏱️ 12 min)

À partir de trois tableaux de même longueur :

```
String[] dishes = {"Margherita", "Tiramisu", "Eau pétillante"};
int[] pricesCents = {1250, 650, 300};
int[] quantities = {2, 1, 3};
```

construisez avec un `StringBuilder` (§4.2), puis affichez, le ticket suivant :

```
2 x Margherita = 25,00 €
1 x Tiramisu = 6,50 €
3 x Eau pétillante = 9,00 €
TOTAL = 40,50 €
```

Réutilisez `formatEuros` de l'exercice 2 du §2.8.

▶ Solution de l'exercice 3

**Exercice 4** 🔵 **— Caractères ou octets ?** (⏱️ 3 min)

Dans un *compact source file*, prédisez puis affichez : `"Crème brûlée".length()` et `"Crème brûlée".getBytes(java.nio.charset.StandardCharsets.UTF_8).length`.

▶ Solution de l'exercice 4

---

# 5. Maven et JUnit

## 5.1 🟢 Pourquoi Maven ?

**Pourquoi ?** `javac` et `java` suffisent pour un fichier. Un vrai projet a des dizaines de classes, des **dépendances** (des bibliothèques écrites par d'autres, comme JUnit), des tests à lancer et une archive à produire. Le faire à la main, c'est taper de longues commandes, télécharger des `.jar` sur le web et oublier une étape.

**Comment ? Maven** est l'outil de construction (*build*) le plus répandu en Java. Il :

- **décrit** le projet dans un seul fichier, le `pom.xml` (*Project Object Model*) ;
- **télécharge** automatiquement les dépendances depuis **Maven Central**, le dépôt public de bibliothèques Java, et les met en cache dans `~/.m2/repository` ;
- **construit** le projet en une commande : compiler, tester, empaqueter ;
- **impose une structure standard** : n'importe quel développeur Java s'y retrouve dans n'importe quel projet Maven. Les IDE (IntelliJ IDEA, VS Code, Eclipse) ouvrent un projet Maven directement à partir de son `pom.xml`.

Gradle est l'autre outil répandu ; les principes sont les mêmes.

## 5.2 🟢 Structure d'un projet

```
foodfast/
├── pom.xml                                  ← description du projet
├── src/
│   ├── main/
│   │   └── java/
│   │       └── fr/foodfast/
│   │           ├── OrderCalculator.java     ← code de l'application
│   │           └── InputValidator.java
│   └── test/
│       └── java/
│           └── fr/foodfast/
│               ├── OrderCalculatorTest.java  ← tests (même package)
│               └── InputValidatorTest.java
└── target/                                  ← généré par Maven (ne pas
versionner)
```

- `src/main/java` : le code livré. `src/test/java` : les tests, jamais livrés.

- `src/main/resources` et `src/test/resources` (facultatifs) : fichiers non Java (configuration, données de test).
- **Packages.** Dans un projet, on range les classes dans un **package**, ici `fr.foodfast`. Le fichier commence alors par la ligne `package fr.foodfast;` et se trouve dans le dossier `fr/foodfast/`. Les tests sont dans le **même package**, dans l'arborescence `src/test/java`.
- `target/` contient les `.class`, les rapports de tests et le `.jar`. Ajoutez-le au `.gitignore`.

> Les exemples de ce support n'ont pas de ligne `package` : c'est ce qui permet de les exécuter seuls, avec `java Fichier.java`. Pour les mettre dans votre projet Maven, ajoutez `package fr.foodfast;` en première ligne.

## 5.3 🟢 Le `pom.xml` minimal (Java 25)

Voici un POM complet et **vérifié** (Maven 3.9, JDK 25, versions publiées sur Maven Central en octobre 2026) :

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <!-- Coordonnées du projet -->
    <groupId>fr.foodfast</groupId>
    <artifactId>foodfast</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <!-- Compile pour Java 25 et vérifie qu'on n'utilise que l'API de Java
25 -->
        <maven.compiler.release>25</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <!-- JUnit 6 : API, moteur et tests paramétrés -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>6.1.3</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.16.0</version>
            </plugin>
            <plugin>
                <!-- Exécute les tests pendant mvn test -->
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.6.0</version>
            </plugin>
        </plugins>
    </build>
</project>
```

Explications :

| Élément | Rôle |
|---|---|
| `groupId` | l'organisation, souvent un nom de domaine inversé (`fr.foodfast`) |
| `artifactId` | le nom du projet (`foodfast`) |
| `version` | `SNAPSHOT` signifie « version en cours de développement » |
| `maven.compiler.release` | version de Java ciblée. `release` (et non `source` / `target`) vérifie aussi que vous n'utilisez pas d'API absente de cette version |
| `project.build.sourceEncoding` | encodage des fichiers source : UTF-8, pour les accents |
| `dependency` | une bibliothèque, identifiée par `groupId:artifactId:version` |
| `<scope>test</scope>` | la dépendance n'est disponible que pour `src/test/java`, et n'est pas livrée |
| `plugin` | un outil de construction. On **épingle** sa version pour un build reproductible |

**Piège : les POM recopiés de vieux tutoriels.**

❌ Trois erreurs fréquentes, parfois dans le même fichier :

```
<maven.compiler.source>11</maven.compiler.source>   <!-- Java 11 : pas de void main(), pas de IO -->
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>5.8.2</version>                         <!-- n'existe pas : JUnit 4 s'arrête à 4.13.2 -->
</dependency>
```

✅ `maven.compiler.release` à 25, et JUnit 5 ou 6 sous le groupe **`org.junit.jupiter`**. Les versions évoluent : vérifiez la dernière sur Maven Central en début de projet. Une dépendance ancienne peut aussi contenir des vulnérabilités connues.

> 🔒 **Sécurité** — Chaque dépendance est du code écrit par d'autres qui s'exécute dans votre application : c'est la **chaîne d'approvisionnement logicielle** (*supply chain*).

> Ajoutez-en le moins possible, épinglez les versions, mettez-les à jour, et méfiez-vous des noms proches des bibliothèques connues (*typosquatting*). Des outils comme OWASP Dependency-Check signalent les dépendances vulnérables.

> 🔵 **Le BOM JUnit.** Si vous ajoutez plusieurs artefacts JUnit (par exemple `junit-platform-launcher` pour lancer des tests depuis du code), importez le *Bill of Materials* `org.junit:junit-bom:6.1.3` dans une section `<dependencyManagement>` (avec `<type>pom</type>` et `<scope>import</scope>`). Vous pouvez alors omettre la version de chaque artefact JUnit : le BOM garantit qu'elles sont cohérentes entre elles.

## 5.4 🟢 Les commandes Maven

Maven organise la construction en **phases** qui s'enchaînent dans un ordre fixe, le **cycle de vie** :

```
validate → compile → test → package → verify → install → deploy
```

Demander une phase exécute **toutes les précédentes**.

| Commande | Effet |
|---|---|
| `mvn compile` | compile `src/main/java` vers `target/classes` |
| `mvn test` | compile le code et les tests, puis exécute les tests |
| `mvn package` | … puis crée `target/foodfast-1.0-SNAPSHOT.jar` |
| `mvn verify` | … puis lance les vérifications supplémentaires (couverture, analyse) |
| `mvn clean` | supprime `target/` ; souvent combiné : `mvn clean test` |
| `mvn -q test` | idem, en mode silencieux (n'affiche que les erreurs) |

Comme `package` passe par `test`, **un test rouge bloque la création du `.jar`** : c'est voulu. Pour exécuter le programme après `mvn compile` :

```
$ java -cp target/classes fr.foodfast.OrderCalculator
```

Le premier lancement télécharge les plugins et dépendances : il faut une connexion internet, et cela peut prendre une minute.

## 5.5 🟢 Premier test JUnit

**Pourquoi ?** Jusqu'ici, on vérifie un programme en lisant ce qu'il affiche. C'est lent, on oublie des cas, et il faut tout revérifier à chaque modification. Un **test automatisé** est un programme qui vérifie un autre programme : il dit vert ou rouge, en quelques millisecondes, autant de fois qu'on veut. Les tests sont votre **filet de sécurité** : modifier un code testé est beaucoup moins risqué.

**Comment ? JUnit** est la bibliothèque de tests standard en Java (version 6 depuis septembre 2025 ; l'API que nous utilisons est la même qu'en JUnit 5). Un test est une méthode annotée `@Test`, dans une classe dont le nom finit par `Test`, placée dans `src/test/java`.

```java
// Fichier : OrderCalculatorTest.java (avec OrderCalculator.java)
import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.Test;

class OrderCalculatorTest {

    @Test
    void total_de_deux_pizzas_et_une_boisson() {
        // Given : une commande de 2 pizzas à 12,50 € et 1 boisson à 3,50 €
        int[] prices = {1250, 350};
        int[] quantities = {2, 1};

        // When : on calcule le total
        int total = OrderCalculator.totalCents(prices, quantities);

        // Then : 2 × 1250 + 350 = 2850
        assertEquals(2850, total);
    }

    @Test
    void une_commande_vide_coute_zero() {
        assertEquals(0, OrderCalculator.totalCents(new int[0], new int[0]));
    }

    @Test
    void une_remise_de_10_pourcent_sur_28_euros_50() {
        assertEquals(2565, OrderCalculator.applyDiscount(2850, 10));
    }

    @Test
    void la_remise_tronque_les_fractions_de_centime() {
        assertEquals(1700, OrderCalculator.applyDiscount(1999, 15));   //
299,85 → 299
    }

    @Test
    void la_livraison_est_offerte_des_30_euros() {
        assertEquals(0, OrderCalculator.deliveryFeeCents(3000, "PERIPHERIE"));
        assertEquals(399, OrderCalculator.deliveryFeeCents(2999,
"PERIPHERIE"));
    }
}
```

Décryptage :

- `@Test` est une **annotation** : une étiquette qui dit à JUnit « cette méthode est un test, exécute-la ».

- `assertEquals(attendu, obtenu)` échoue si les deux valeurs diffèrent. L'ordre compte pour le message d'erreur : la valeur **attendue** d'abord.
- `import static` permet d'écrire `assertEquals(...)` au lieu de `Assertions.assertEquals(...)`.
- La classe et les méthodes de test n'ont pas besoin d'être `public`. Elles ne doivent pas être `private`.
- Le test appelle `OrderCalculator.totalCents(...)` : il faut donc une **classe nommée** (§1.4).

Autres assertions courantes : `assertTrue(condition)`, `assertFalse(condition)`, `assertNotNull(valeur)`, `assertArrayEquals(tableauAttendu, tableauObtenu)`.

**Exécuter.** `mvn test` compile et lance tous les tests :

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Remplacez `Math.multiplyExact` par `Math.addExact` dans `totalCents` et relancez : le test devient rouge et explique pourquoi.

```
[ERROR] Failures:
[ERROR]   OrderCalculatorTest.total_de_deux_pizzas_et_une_boisson:19 expected:
<2850> but was: <1603>
[ERROR] Tests run: 5, Failures: 1, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
```

Le rapport donne la classe, la méthode, la ligne du test (`:19`) et l'écart entre l'attendu et l'obtenu. Remettez `Math.multiplyExact` : tout redevient vert.

**Piège : le test qui ne s'exécute jamais.**

❌ La classe s'appelle `CheckOrder` : par défaut, Maven (surefire) n'exécute que les classes dont le nom finit par `Test`, `Tests` ou `TestCase`, ou commence par `Test`. Résultat : `Tests run: 0` … et un `BUILD SUCCESS` trompeur. Même symptôme si le test est rangé dans `src/main/java`.

✅ Nom en `…Test`, dans `src/test/java`. Et vérifiez toujours que le **nombre de tests exécutés** augmente quand vous en ajoutez un. Mieux : voyez chaque nouveau test échouer au moins une fois (cassez volontairement le code), pour être sûr qu'il teste vraiment quelque chose.

## 5.6 🟢 Écrire de bons tests

**La structure Given / When / Then** (ou *Arrange* / *Act* / *Assert*) rend un test lisible par tous :

- **Given** (étant donné) : on prépare les données.
- **When** (quand) : on exécute **une** action, celle qu'on teste.
- **Then** (alors) : on vérifie le résultat.

**Quoi tester ?**

- Le **cas nominal** : l'usage normal.
- Les **cas limites**, là où se cachent les bugs : `0`, `1`, la borne maximale (`20`) et juste au-dessus (`21`), tableau vide, chaîne vide.
- Les **entrées invalides** : `null`, `"abc"`, `"-2"`. Pour un profil cyber, ce sont des **tests de sécurité**.
- Pas le JDK ni les bibliothèques : inutile de tester que `Math.max` fonctionne.

**Les qualités d'un bon test (F.I.R.S.T.)** : *Fast* (rapide), *Independent* (ne dépend pas d'un autre test), *Repeatable* (même résultat à chaque exécution), *Self-validating* (vert ou rouge, sans lire de logs), *Timely* (écrit en même temps que le code, voire avant).

**Le nom du test décrit le comportement attendu.** `test1` ne dit rien ; `une_quantite_de_21_est_refusee` documente la règle métier.

```java
// Fichier : InputValidatorTest.java (avec InputValidator.java)
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

import org.junit.jupiter.api.Test;

class InputValidatorTest {

    @Test
    void une_quantite_de_3_est_acceptee() {
        assertTrue(InputValidator.isValidQuantity("3"));
    }

    @Test
    void les_bornes_1_et_20_sont_acceptees() {
        assertTrue(InputValidator.isValidQuantity("1"));
        assertTrue(InputValidator.isValidQuantity("20"));
    }

    @Test
    void une_quantite_de_0_ou_21_est_refusee() {
        assertFalse(InputValidator.isValidQuantity("0"));
        assertFalse(InputValidator.isValidQuantity("21"));
    }

    @Test
    void une_saisie_absente_est_refusee() {
        assertFalse(InputValidator.isValidQuantity(null));
        assertFalse(InputValidator.isValidQuantity(""));
        assertFalse(InputValidator.isValidQuantity("   "));
    }

    @Test
    void une_saisie_non_numerique_est_refusee() {
        assertFalse(InputValidator.isValidQuantity("abc"));
        assertFalse(InputValidator.isValidQuantity("-2"));
        assertFalse(InputValidator.isValidQuantity("٣"));   // chiffre arabe-
indien
    }

    @Test
    void un_tres_grand_nombre_est_refuse_sans_planter() {
        assertFalse(InputValidator.isValidQuantity("999999999999"));
    }
}
```

**Piège : un test qui ne vérifie rien.**

❌ Ce test est toujours vert, quel que soit le code :

```
@Test
void total() {
    OrderCalculator.totalCents(new int[] {1250}, new int[] {2});   // aucune
assertion
}
```

✅ Chaque test contient au moins une assertion sur le résultat attendu.

## 5.7 🔵 Tests paramétrés et tests d’exception

**Tests paramétrés.** Quand on teste la même règle sur de nombreuses valeurs, `@ParameterizedTest` évite de répéter le test. Chaque valeur devient un cas affiché séparément dans le rapport.

```java
// Fichier : QuantityParameterizedTest.java (avec InputValidator.java)
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.NullAndEmptySource;
import org.junit.jupiter.params.provider.ValueSource;

class QuantityParameterizedTest {

    @ParameterizedTest
    @NullAndEmptySource
    @ValueSource(strings = {" ", "0", "21", "-2", "abc", "3a", " 3", "3.5",
"999999999999", "٢"})
    void les_saisies_invalides_sont_refusees(String input) {
        assertFalse(InputValidator.isValidQuantity(input));
    }

    @ParameterizedTest
    @CsvSource({
            "1, true",
            "20, true",
            "21, false",
            "07, true"
    })
    void la_validite_depend_des_bornes(String input, boolean expected) {
        assertEquals(expected, InputValidator.isValidQuantity(input));
    }
}
```

**Tester qu'une exception est levée.** `assertThrows` vérifie qu'un appel échoue avec le bon type d'exception. Il utilise la syntaxe `() -> …`, une **lambda**, que nous étudierons en séance 3. Pour l'instant, lisez-la comme « le code à exécuter » :

```java
// Fichier : OrderCalculatorErrorTest.java (avec OrderCalculator.java)
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.Test;

class OrderCalculatorErrorTest {

    @Test
    void une_remise_superieure_a_100_est_refusee() {
        assertThrows(IllegalArgumentException.class,
                () -> OrderCalculator.applyDiscount(2850, 150));
    }

    @Test
    void un_total_qui_deborde_est_detecte() {
        int[] prices = {1250};
        int[] quantities = {2_000_000};
        assertThrows(ArithmeticException.class,
                () -> OrderCalculator.totalCents(prices, quantities));
    }
}
```

🔵 **Couverture de code.** L'outil JaCoCo mesure quelles lignes et quelles branches vos tests exécutent. Pour Java 25, il faut **JaCoCo 0.8.14 ou plus récent** : les versions antérieures ne savent pas lire le bytecode de Java 25. Une couverture élevée ne garantit pas de bons tests (voir le piège « test qui ne vérifie rien ») ; elle sert surtout à repérer le code **jamais** testé.

## 5.8 🛠️ À vous

⏱️ 30 à 40 min, sur votre machine. C'est l'atelier le plus important de la séance : le TP commence exactement comme cela.

**Exercice 1 — Un projet Maven qui teste** (⏱️ 15 min)

1. Reprenez le projet IntelliJ du §0.3 (ou créez-en un : *New Project*, build system **Maven**, JDK 25).
2. Remplacez le contenu du `pom.xml` par celui du §5.3, en gardant votre `groupId` et votre `artifactId`. Cliquez sur l'icône Maven qui apparaît en haut à droite de l'éditeur (*Load Maven Changes*) pour qu'IntelliJ relise le fichier.

3. Copiez `BasketStats` (exercice 3 du §3.9) dans `src/main/java`. Vous ne l'avez pas fait ? Commencez avec la seule méthode `countAbove`, donnée sur la slide « À vous — Partie 5 », et ne gardez que le test du cas limite.
4. Dans `src/test/java`, créez `BasketStatsTest` avec trois tests :
   - un cas nominal : le maximum de `{1250, 450, 800, 1590}` vaut `1590` ;
   - un cas limite : `countAbove` avec un prix **égal** au seuil ne le compte pas ;
   - une entrée invalide : `averageCents` sur un tableau vide lève une `IllegalArgumentException`.
5. Lancez les tests de **deux** façons : dans un terminal, `mvn test` à la racine du projet ; dans IntelliJ, la double flèche verte ▶▶ à côté du nom de la classe de test.

**Indice.** Raccourci IntelliJ : placez le curseur sur le nom de la classe `BasketStats`, Alt+Entrée → *Create Test*. IntelliJ crée la classe au bon endroit.

▶ Solution de l'exercice 1

**Exercice 2 — Casser exprès, et lire le rapport** (⏱ 8 min)

1. Dans `BasketStats.countAbove`, remplacez `price > thresholdCents` par `price >= thresholdCents`.
2. Relancez `mvn test`. Lisez le rapport : quel test échoue ? Que signifient `expected` et `but was` ?
3. Rétablissez le code et vérifiez que tout redevient vert.

▶ Solution de l'exercice 2

**Exercice 3** 🔵 **— Tester d'abord** (⏱ 10 min)

Écrivez d'abord un test paramétré (§5.7) pour `parseQuantity` (exercice 2 du §4.8) avec `@CsvSource` : chaque ligne contient une saisie et le résultat attendu (`"3, 3"`, `"abc, -1"`, `"21, -1"`…). Lancez-le **avant** d'avoir copié `parseQuantity` dans le projet : il ne compile pas (rouge). Copiez la méthode dans une classe `QuantityParser` : il passe (vert).

**Indice.** Dans `@CsvSource`, entourez une valeur d'apostrophes pour garder ses espaces : `"' 3', -1"`.

---

# 6. 🟣 Défis pour ceux qui connaissent déjà

Vous avez déjà programmé en Java ? Pendant que le reste du groupe consolide les bases, voici sept défis (§6.1 à §6.6 et §6.9), et deux sections pour mettre à jour vos connaissances (§6.7 et §6.8). Chaque défi se fait en 15 à 40 minutes. Écrivez d'abord les **tests JUnit** (dans un projet Maven), puis le code. Les solutions sont dans l'annexe A, repliées : ne les ouvrez qu'après avoir essayé.

## 6.1 🟣 Défi 1 — Algorithme de Luhn (numéro de carte)

**Contexte.** Avant d'envoyer un numéro de carte bancaire au prestataire de paiement, FoodFast vérifie qu'il n'y a pas de faute de frappe. Les numéros de carte contiennent une **clé de contrôle** calculée par l'algorithme de Luhn.

**Énoncé.** Écrivez `static boolean isValidCardNumber(String number)` :

1. Les espaces sont autorisés et ignorés (`"4111 1111 1111 1111"`). Tout autre caractère non chiffre rend le numéro invalide.
2. Après retrait des espaces, le numéro contient entre 12 et 19 chiffres.
3. En partant de la **droite**, doublez un chiffre sur deux (le 2e, le 4e…). Si un doublement dépasse 9, soustrayez 9. Faites la somme de tous les chiffres : le numéro est valide si la somme est un multiple de 10.

Cas de test : `"4111 1111 1111 1111"` et `"4539 1488 0343 6467"` sont valides ; `"4111 1111 1111 1112"`, `"4111-1111-1111-1111"`, `""` et `null` ne le sont pas.

**Indice.** `c - '0'` convertit un caractère chiffre en sa valeur entière. Parcourez la chaîne de la fin vers le début avec un booléen qui alterne à chaque chiffre.

> 🔒 Luhn détecte les fautes de frappe, **pas** la fraude : n'importe qui peut fabriquer un numéro valide au sens de Luhn. Et un vrai numéro de carte ne doit jamais apparaître dans vos logs (norme PCI DSS).

## 6.2 🟣 Défi 2 — Robustesse d'un mot de passe

**Énoncé.** Écrivez `static String passwordStrength(String password)` qui renvoie `"REFUSÉ"`, `"FAIBLE"`, `"MOYEN"` ou `"FORT"` :

- `"REFUSÉ"` si le mot de passe est `null`, fait moins de 8 caractères, ou figure (sans tenir compte de la casse) dans une petite liste de mots de passe courants : `password`, `motdepasse`, `azerty123`, `123456789`, `foodfast2026`.
- Sinon, comptez un point par critère : longueur ≥ 12 ; longueur ≥ 16 ; présence d'une minuscule ; d'une majuscule ; d'un chiffre ; d'un caractère qui n'est ni lettre ni chiffre. Score ≤ 2 : `"FAIBLE"` ; 3 ou 4 : `"MOYEN"` ; 5 ou 6 : `"FORT"`.

**Indice.** `Character.isLowerCase`, `isUpperCase`, `isDigit`, `isLetterOrDigit`. Pour la liste, un tableau de `String` et `equalsIgnoreCase`.

> 🔒 **Pour aller plus loin.** Les recommandations actuelles (NIST SP 800-63B, ANSSI) privilégient la **longueur** et la vérification contre des listes de mots de passe **compromis**, plutôt que des règles de composition imposées (« une majuscule, un chiffre… ») qui produisent des mots de passe prévisibles comme `Azerty123!`. Testez votre fonction avec cet exemple : qu'en pensez-vous ? Un mot de passe ne se stocke jamais en clair : nous verrons le hachage en séance 4.

## 6.3 🟣 Défi 3 — Chiffre de César

**Énoncé.**

1. Écrivez `static String caesar(String text, int shift)` qui décale chaque lettre de `shift` positions dans l'alphabet, en conservant la casse et en laissant intacts les autres caractères (espaces, ponctuation, chiffres, lettres accentuées). `caesar("Pizza, 2 parts !", 3)` donne `"Slccd, 2 sduwv !"`. Le décalage peut être négatif ou supérieur à 26.
2. Vérifiez que `caesar(caesar(t, k), -k)` redonne `t`.
3. Écrivez `static void bruteForce(String cipherText)` qui affiche les 26 déchiffrements possibles. Retrouvez le message de `"Cvayl jvtthukl lza wylal"`.

**Indice.** Pour une minuscule `c` : `(char) ('a' + Math.floorMod(c - 'a' + shift, 26))`. Pourquoi `Math.floorMod` et pas `%` ? Essayez avec `shift = -3` et la lettre `a`.

> 🔒 César se casse en 26 essais : c'est un exercice de manipulation de caractères, pas de la cryptographie. En pratique, on utilise les algorithmes standards du JDK (`javax.crypto`), jamais un chiffrement « maison ».

## 6.4 🟣 Défi 4 — FizzBuzz piloté par les tests (TDD)

**Énoncé.** Pratiquez le **TDD** (*Test-Driven Development*) : on écrit un test qui échoue (**rouge**), puis le minimum de code pour le faire passer (**vert**), puis on améliore le code sans casser les tests (**refactor**). Recommencez avec le test suivant.

Écrivez `static String fizzBuzz(int n)` en suivant **strictement** cet ordre de tests, un à la fois :

1. `fizzBuzz(1)` renvoie `"1"`, `fizzBuzz(2)` renvoie `"2"`.
2. `fizzBuzz(3)` renvoie `"Fizz"` ; `fizzBuzz(6)` aussi.
3. `fizzBuzz(5)` renvoie `"Buzz"` ; `fizzBuzz(10)` aussi.
4. `fizzBuzz(15)` renvoie `"FizzBuzz"` ; `fizzBuzz(30)` aussi.
5. `fizzBuzz(0)` et `fizzBuzz(-3)` lèvent une `IllegalArgumentException`.

Variante FoodFast : la commande numéro `n` reçoit un dessert offert si `n` est multiple de 3, une boisson si multiple de 5, les deux si multiple de 15.

**Indice.** L'ordre des conditions compte : que se passe-t-il si vous testez `n % 3 == 0` avant `n % 15 == 0` ? Un `@ParameterizedTest` avec `@CsvSource` (§5.7) rend les tests très compacts.

## 6.5 🟣 Défi 5 — `switch` à motifs sur un `record`

**Contexte.** Java permet de déclarer en une ligne un type de données immuable, le **`record`** (séance 2), et de « déconstruire » ses composants dans un `switch` (*record patterns*, Java 21). Avec une interface **`sealed`**, le compilateur connaît la liste exhaustive des cas.

**Énoncé.** FoodFast accepte trois moyens de paiement :

```
sealed interface Payment permits CardPayment, VoucherPayment, CashPayment {}
record CardPayment(String last4, int amountCents) implements Payment {}
record VoucherPayment(String code, int amountCents) implements Payment {}
record CashPayment(int amountCents) implements Payment {}
```

Écrivez `static int processingFeeCents(Payment payment)` avec un **seul** `switch` en expression, **sans** **`default`** :

- carte : 1,5 % du montant, tronqué ;
- bon d'achat : gratuit ;
- espèces : 1 € si le montant dépasse 50 €, sinon gratuit (utilisez une garde `when`).

Puis écrivez `static String describe(Payment payment)` qui renvoie par exemple `"Carte ****1234 : 25,00 €"`.

Ajoutez un quatrième type de paiement `record MealVoucher(...)` à la liste `permits` : que dit le compilateur ? Pourquoi est-ce une bonne nouvelle ?

**Indice.** `case CardPayment(String last4, int amount) -> …` ; `case CashPayment(int amount) when amount > 5000 -> …`. L'ordre des `case` compte quand il y a une garde.

## 6.6 🟣 Défi 6 — Générer un code promo imprévisible

**Énoncé.** Écrivez `static String generatePromoCode(int length)` qui produit un code de `length` caractères (entre 6 et 16, sinon `IllegalArgumentException`) choisis dans l'alphabet `ABCDEFGHJKLMNPQRSTUVWXYZ23456789` (sans `I`, `O`, `0`, `1`, faciles à confondre). Le code doit passer `InputValidator.isValidPromoCode` (§4.3).

Questions :

1. Pourquoi faut-il utiliser `java.security.SecureRandom` et non `java.util.Random` ou `Math.random()` ?
2. Combien de codes différents existent pour une longueur de 8 ? Combien d'essais en moyenne pour un attaquant qui tente des codes au hasard, s'il y a 1 000 codes valides en circulation ?
3. Quelle protection côté serveur rend cette attaque impraticable ?

**Indice.** `random.nextInt(alphabet.length())` donne un indice aléatoire. Un `StringBuilder` pour construire le code.

## 6.7 🔵 Java moderne : si vous avez connu Java 8 ou 11

Vous avez appris Java il y a quelques années, ou vous travaillez sur un projet en Java 8 ou 11 ? Voici ce qui a changé, et ce que ce cours utilise. Chaque ligne est détaillée dans la séance indiquée.

| Avant | Aujourd'hui (Java 25) | Depuis | Séance |
|---|---|---|---|
| `public class X { public static void main(String[] args) … }` pour tout | `void main()` dans un *compact source file*, `IO.println` | 25 | 1 |

| Avant | Aujourd'hui (Java 25) | Depuis | Séance |
|---|---|---|---|
| `Map<String, List<Order>> m = new HashMap<String, List<Order>>();` | `var m = new HashMap<String, List<Order>>();` | 10 | 1 |
| `switch` avec `case …: … break;` | `switch` en expression avec `->`, sans `break` | 14 | 1 |
| Chaînes multi-lignes avec `+` et `\n` | *Text blocks* `"""…"""` | 15 | 1 |
| Classe de données : champs, constructeur, `equals`, `hashCode`, `toString` à la main (ou Lombok) | `record Dish(String name, BigDecimal price) {}` | 16 | 2 |
| `if (o instanceof Dish) { Dish d = (Dish) o; … }` | `if (o instanceof Dish d) { … }` | 16 | 2 |
| Hiérarchie ouverte à tous | `sealed interface … permits …` | 17 | 2 |
| `switch` sur des constantes seulement | `switch` sur des types, avec déconstruction des `record` et gardes `when` | 21 | 2 |
| `list.get(0)`, `list.get(list.size() - 1)` | `list.getFirst()`, `list.getLast()` | 21 | 2 |
| `stream.collect(Collectors.toList())` | `stream.toList()` (non modifiable) | 16 | 3 |
| Pool de threads limité, code asynchrone compliqué | *Virtual threads* : un thread par tâche, même pour 10 000 tâches | 21 | 4 |
| `import java.util.*; import java.io.*; …` | `import module java.base;` | 25 | 1 |

**Ce qu'il faut désapprendre :**

- `new Integer(5)`, `new Double(…)` : dépréciés et appelés à disparaître (le compilateur vous prévient). Écrivez `5`, ou `Integer.valueOf(5)`.

- `Date`, `Calendar`, `SimpleDateFormat` : remplacés par `java.time` (`LocalDate`, `Instant`…), séance 2.
- `Vector`, `Hashtable`, `StringBuffer` : remplacés par `ArrayList`, `HashMap`, `StringBuilder`.
- `e.printStackTrace()` pour « gérer » une erreur : séance 3.
- Le JRE séparé : il n'existe plus, voir §1.1.

## 6.8 🟣 Sous le capot : quatre manipulations

Ces manipulations ne sont pas nécessaires pour le TP. Elles expliquent des comportements qui surprennent même des développeurs expérimentés.

### 1. Le cache des `Integer`.

```
// Fichier : IntegerCache.java
void main() {
    Integer a = 127, b = 127;
    IO.println(a == b);       // true
    Integer c = 128, d = 128;
    IO.println(c == d);       // false
}
```

`Integer` est une **classe** (le type « emballé » de `int`). `==` compare donc des **références**, comme pour les `String`. Java garde en cache les objets `Integer` de -128 à 127 : `a` et `b` désignent le même objet, `c` et `d` deux objets différents. Morale : `equals` pour comparer des `Integer`, toujours. Ce bug passe tous les tests avec de petites valeurs, puis apparaît en production.

### 2. Le *string pool*.

```
// Fichier : StringPool.java
void main() {
    String s1 = "PROMO10";
    String s2 = new String("PROMO10");
    IO.println(s1 == s2);            // false
    IO.println(s1 == s2.intern());   // true
}
```

Les littéraux identiques sont partagés dans un *pool*. `intern()` renvoie l'exemplaire du pool. C'est pourquoi `==` sur des `String` « marche parfois » (§2.5) : un piège, pas une fonctionnalité.

**3. Lire le bytecode d'une méthode.** Compilez `OrderCalculator` (§3.5), puis lancez `javap -c OrderCalculator`. Dans `totalCents`, retrouvez la boucle : `if_icmpge` (sortir si `i >= length`), `iaload` (lire une case du tableau), les deux appels `invokestatic` à `Math.multiplyExact` et `Math.addExact`, `iinc` (le `i++`) et `goto` (revenir au test). Remplacez ensuite `Math.addExact(total, line)` par `total + line`, recompilez : l'appel est remplacé par une seule instruction `iadd`, qui ne vérifie rien.

**4. Le JIT au travail.** Ce programme mesure dix fois le même calcul :

```
// Fichier : JitWarmup.java
static long sumOfSquares(int n) {
    long sum = 0;
    for (int i = 0; i < n; i++) {
        sum += (long) i * i;
    }
    return sum;
}

void main() {
    for (int round = 1; round <= 10; round++) {
        long start = System.nanoTime();
        long result = sumOfSquares(5_000_000);
        long micros = (System.nanoTime() - start) / 1_000;
        IO.println("Tour " + round + " : " + micros + " µs (" + result + ")");
    }
}
```

Les premiers tours sont nettement plus lents : la JVM **interprète** d'abord la méthode, puis la compile en code machine quand elle devient « chaude ». Relancez avec `java -Xint JitWarmup.java` (interprétation seule, sans JIT) et comparez. C'est pourquoi un *benchmark* Java sérieux ne mesure jamais les premières exécutions (outil de référence : JMH).

## 6.9 🟣 Défi 7 — Parler le protocole Redis à la main

**Contexte.** Le TP consiste à écrire un serveur compatible avec Redis et Valkey. Ils communiquent avec le protocole **RESP**, un format texte très simple. Une commande `GET nom` est envoyée sous la forme d'un tableau de deux chaînes :

```
*2\r\n$3\r\nGET\r\n$3\r\nnom\r\n
```

`*2` annonce un tableau de 2 éléments ; `$3` annonce une chaîne de **3 octets** ; `\r\n` termine chaque ligne.

**Énoncé.**

1. Écrivez `static String encodeCommand(String... words)` qui produit cette représentation. `encodeCommand("GET", "nom")` doit renvoyer exactement la chaîne ci-dessus.
2. Attention aux accents : `encodeCommand("SET", "plat", "crème")` doit annoncer `$6` pour `crème` (6 octets en UTF-8), et non `$5`.
3. Si Docker est installé, lancez un vrai serveur Valkey (`docker run --rm -p 6379:6379 valkey/valkey:8`) et envoyez-lui votre commande avec `printf '…' | nc localhost 6379`. Il doit répondre `+OK`.

**Indice.** `String... words` (paramètre « varargs ») se manipule comme un tableau `String[]`. Pour la longueur en octets : §4.7.

Le starter du TP fournit une classe `RespWriter` qui fait ce travail : comparez votre version à la sienne.

---

## 7. Ce qu'il faut retenir

- **Chaîne d'exécution** : `.java` → `javac` → bytecode `.class` → JVM (interprétation puis JIT, ramasse-miettes). On installe le **JDK 25**.
- **Premier programme** : `void main()` et `IO.println` dans un *compact source file*. Dans un projet : une **classe nommée** (`public class … public static void main(String[] args)`), seule forme appelable et donc testable.
- **Types** : primitifs (`int`, `long`, `double`, `boolean`, `char`) et références (`String`, qui peut valoir `null`). `var` déduit le type à la compilation, il ne le rend pas dynamique.
- **Argent** : pas de `double`. Aujourd'hui des centimes dans un `int`, en séance 2 `BigDecimal`.
- **Opérateurs** : division entière tronquée, `%` pour le reste, `&&` / `||` en court-circuit, parenthèses dans les concaténations.
- **`String`** : comparer avec `equals`, jamais `==`. Immuable : récupérer le résultat des méthodes. `StringBuilder` dans les boucles.
- **Contrôle** : `if` / `else` avec accolades, `switch` en expression avec `->`, `for`, for-each, `while`. Tableaux : indices de `0` à `length - 1`.
- **Méthodes `static`** : un nom, des paramètres, un `return`. Vérifier les paramètres avant de calculer.

- 🔒 **Débordement** : un `int` déborde en silence. `Math.addExact` / `multiplyExact`, et des entrées bornées.
- 🔒 **Validation** : présence → format (liste blanche, longueur) → bornes → conversion.
- **Conversion** : `Long.parseLong` lève `NumberFormatException` ; un `try` / `catch` précis évite l'arrêt du programme.
- **Lire `List` et `Map`** : `size`, `get`, for-each ; `put`, `get` (qui renvoie `null` si absent), `getOrDefault`, `containsKey`, `remove`.
- 🔵 **Octets** : en UTF-8, `"é".length()` vaut 1 mais `getBytes(UTF_8).length` vaut 2.
- **Maven** : `pom.xml` avec `maven.compiler.release` 25, `org.junit.jupiter:junit-jupiter` en `test`, surefire épinglé. `mvn test`.
- **JUnit** : `@Test`, `assertEquals(attendu, obtenu)`, Given / When / Then, cas nominal, limites et entrées invalides.

---

## 8. Pour aller plus loin

- dev.java — Learn : le parcours officiel d'apprentissage de Java.
- JEP 512 — Compact Source Files and Instance Main Methods et JEP 511 — Module Import Declarations.
- JEP 361 — Switch Expressions et JEP 378 — Text Blocks.
- Documentation Java 25 : `String`, `StringBuilder`, `Math`, `IO`.
- Maven in 5 Minutes et le guide de l'utilisateur JUnit.
- 🔒 SEI CERT Oracle Coding Standard for Java : règles NUM00-J (débordement d'entier) et IDS (validation des entrées).
- 🔒 OWASP Input Validation Cheat Sheet et CWE-190.

---

## Annexe A — Solutions des défis

▶ Solution du défi 1 — Algorithme de Luhn

▶ Solution du défi 2 — Robustesse d'un mot de passe

▶ Solution du défi 3 — Chiffre de César

▶ Solution du défi 4 — FizzBuzz piloté par les tests

▶ Solution du défi 5 — switch à motifs sur un record

▶ Solution du défi 6 — Code promo imprévisible

▶ Solution du défi 7 — Parler le protocole Redis à la main

---

# Annexe B — 🛠️ Lire les erreurs : les plus fréquentes

Un message d'erreur Java dit presque toujours **où** et **quoi**. Lisez la **première** erreur, à la ligne indiquée ; les suivantes en sont souvent la conséquence.

**Erreurs de compilation** ( `javac` , ou soulignées en rouge dans IntelliJ) :

| Message | Cause habituelle | Correction |
|---|---|---|
| `';' expected` | point-virgule oublié en fin d'instruction | ajouter le `;` à la position du `^` (ou juste avant) |
| `cannot find symbol` … `symbol: variable total` | variable mal orthographiée, ou déclarée dans un autre bloc (§3.7) | vérifier l'orthographe et la portée |
| `cannot find symbol` … `symbol: method printn(String)` | nom de méthode mal écrit, ou mauvais types d'arguments | vérifier le nom ; Ctrl+Espace propose les méthodes existantes |
| `cannot find symbol` … `symbol: class List` | `import` manquant (dans une classe nommée) | Alt+Entrée → *Import class* |
| `incompatible types: String cannot be converted to int` | `int quantity = "3";` | convertir : `Integer.parseInt("3")` (§4.6) |
| `incompatible types: possible lossy conversion from double to int` | `int cents = 12.5 * 100;` | décider de l'arrondi, puis *cast* explicite (§2.7) |
| `variable total might not have been initialized` | variable lue avant d'avoir reçu une valeur | l'initialiser à la déclaration ( `int total = 0;` ) |
| `missing return statement` | un chemin de la méthode ne se termine pas par `return` | ajouter le `return` du cas manquant (souvent le `else` ) |
| `unclosed string literal` | guillemet fermant oublié | fermer le texte sur la même ligne |
| `reached end of file while parsing` | accolade fermante `}` oubliée | Ctrl+Alt+L (reformater) montre où l'indentation déraille |

| Message | Cause habituelle | Correction |
|---|---|---|
| `class Order is public, should be declared in a file named Order.java` | nom du fichier différent du nom de la classe publique | renommer le fichier, ou la classe (Maj+F6) |
| `non-static method fee() cannot be referenced from a static context` | appel d'une méthode sans `static` depuis `main` | déclarer la méthode `static` (§3.5) ; les objets arrivent en séance 2 |

**Erreurs à l'exécution** (le programme compile, puis s'arrête) :

| Message | Cause habituelle | Correction |
|---|---|---|
| `NullPointerException: Cannot invoke "String.length()" because "name" is null` | une variable vaut `null` (§2.2) | vérifier la provenance de la valeur ; tester `null` aux frontières |
| `ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3` | indice `length` utilisé : les indices vont de `0` à `length - 1` (§3.4) | `i < t.length`, pas `i <= t.length` |
| `NumberFormatException: For input string: "12a"` | texte non numérique passé à `parseInt` (§4.6) | valider avant, ou `try` / `catch` |
| `ArithmeticException: / by zero` | division entière par zéro | tester le diviseur avant |
| `ArithmeticException: integer overflow` | `Math.addExact` a détecté un débordement (§2.6) | c'est voulu : borner les entrées |
| `Erreur : impossible de trouver ou de charger la classe principale Hello` | `java Hello` lancé dans le mauvais dossier, ou `javac` pas encore exécuté | `ls` pour vérifier que `Hello.class` est là |

**Lire une pile d'appels** (*stack trace*) :

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3
out of bounds for length 3
    at BasketStats.maxCents(BasketStats.java:7)
    at BasketStats.main(BasketStats.java:41)
```

La première ligne donne le **type** d'erreur et son **message**. Les lignes `at` remontent les appels : l'erreur s'est produite dans `maxCents`, ligne 7, elle-même appelée par `main`, ligne 41. Commencez par la **première ligne `at` qui concerne votre code**. Dans IntelliJ, cliquez sur `BasketStats.java:7` pour y aller directement.

> 🔵 Le nom de la variable dans le message de `NullPointerException` (`because "name" is null`) n'apparaît que si le code a été compilé avec les informations de débogage, ce que font IntelliJ et Maven. Avec `java Fichier.java`, vous verrez `"<local1>"` à la place.

---

## Annexe C — 🛠️ Pense-bête : vous venez de Python ou de C

| Notion | Python | C | Java |
|---|---|---|---|
| Afficher | `print("x", n)` | `printf("x %d\n", n);` | `IO.println("x " + n);` |
| Variable | `n = 3` | `int n = 3;` | `int n = 3;` ou `var n = 3;` |
| Type vérifié | à l'exécution | à la compilation | à la compilation |
| Bloc | indentation | `{ … }` | `{ … }` (indentez quand même) |
| Fin d'instruction | fin de ligne | `;` | `;` |
| Texte | `"x"` ou `'x'` | `char*` | `"x"` est une `String`, `'x'` est un `char` |
| Comparer deux textes | `a == b` | `strcmp(a, b) == 0` | `a.equals(b)`, **jamais** `a == b` |
| Et, ou, non | `and`, `or`, `not` | `&&`, `\|\|`, `!` | `&&`, `\|\|`, `!` |

| Notion | Python | C | Java |
|---|---|---|---|
| Division entière | `7 // 2` | `7 / 2` | `7 / 2` (entre deux `int`) |
| Entiers | taille illimitée | débordent (comportement indéfini) | débordent **en silence** (§2.6) |
| Liste | `[1, 2]` | tableau, taille fixe | `int[] t = {1, 2};` (taille fixe), `List.of(1, 2)` |
| Dictionnaire | `{"a": 1}` | — | `Map.of("a", 1)` |
| Boucle sur une liste | `for x in items:` | `for (i = 0; i < n; i++)` | `for (var x : items)` |
| Absence de valeur | `None` | `NULL` | `null` |
| Fonction | `def f(x):` | `int f(int x) { … }` | `static int f(int x) { … }` |
| Mémoire | automatique | `malloc` / `free` | automatique (ramasse-miettes) |
| Lancer | `python f.py` | `gcc f.c && ./a.out` | `java F.java`, ou `javac` puis `java F` |